Week 6

React Fundamentals

The website development process

- Discovery and planning
  - What is the problem?
  - What are we doing to address it?
- Development
  - Actually doing the things
- Deployment
  - Shipping
- Maintenance
  - Keeping up and measuring success

You are now capable of doing a complete app!

From this point on, we're refining our approach

In this lesson:

- What is React, and why to use it
- Starting a simple React project
- Components
- State
- Hooks
- Installing packages
- Some considerations for apps that use servers

You can use the code here as reference for this class:
https://github.com/sgenius/sample-react-2 .
(This may not be exactly the same as in the slides or what we do in class, but it's close.)

What is React and why to use it

What is React?

- React advertises itself as a library to build components in JavaScript.
  - These components are very open-ended, which is why they say it's not "a framework"
- It's also an architecture that implements the composition paradigm
  - The "component" is the building block. Components can contain other components
  - A component is a self-contained part of your program, with defined inputs and outputs
- Comes with a set of "interesting" things to learn
  - Outputs are "JSX", which looks like HTML
  - "Unidirectional data flow": Data always flows from the JS to the HTML. When there is a data input, the JS re-renders instead
    - This means data is much simpler to manage, once you're familiar with how this works
- React apps are compiled! They need Node.js to be hosted and compiled
  - The app can run in the browser, in Node, or in both
- Because this is a node.js app, use packages from the internet to extend your application! (This is also true of other frameworks)
  - Packages contain code that you can download, install, and use in your app.
  - Directory of packages in npmjs.com (use the search bar)

Why bother with learning React?

- Currently, it's THE standard for UX/UI building.
  - Access to jobs!
- It's stable and widely available
  - It's been there for a while: many people use it, so it's not going away soon
  - Originally created by Meta (Facebook); supported by big names in the industry.
  - It's open source, so nobody will start asking you to pay for its use
  - Documentation is very robust and it's easy to get started with it
- It's built for efficiency and speed
  - While mastering it takes some effort, it provides good results from the start
  - Can compile and output to the server for even more efficiency
- Can be used on the web or in mobile apps!
  - Porting apps between one and the other takes minimal effort compared with alternatives

What to learn if I don’t like React?

Alternatives include Vue.js and Angular.js.

- Vue.js is a framework similar to React.
  - It’s mostly meant as a UI building framework
  - It looks and behaves more like a traditional web app, so it can be simpler to learn from beginners
  - Data updates are bidirectional: there are “observers” to update data when the user inputs, instead of re-rendering everything
    - Apps with lots of interactions between data can become more complex to build and maintain
  - Less people use it, so there are less plugins, help, and support in general
  - Also, less jobs available for it
    - In China, however, Vue is more popular than React

What to learn if I don’t like React?

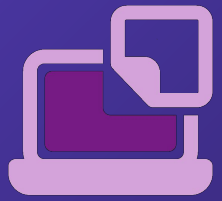
Alternatives include Vue.js and Angular.js.

- Angular.js is a much more complex framework, originally made by Google.
  - Angular was the first real frontend framework.
  - It contains pieces to do EVERYTHING in a frontend app, from data load and state management up to rendering
  - It’s opinionated: Angular is very rigid in how things should be done
    - This means more concepts to learn
    - But also means all Angular apps are relatively similar; become an expert and other apps are easier to understand
  - Support is not as strong as React, but still maintains a healthy community. It’s not going away soon

How to get started?

- The “I just want to learn” way: using a playground or with local files
- The “real” way: install everything in your computer and create a new project

We will get started with the first and progress to the 2nd.



Created by HideMaru
from Noun Project

How to get started?

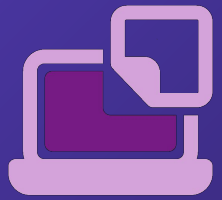
The easiest way to learn React is just by importing it to a page. This will compile locally in the browser. Let's do that now:

- In a new folder in your computer, create an `index.html` page.
- Inside <body>, create a `<div id="root"></div>`
- Include the following tags at the end of <body> (not on <head>; we're cheating a little):

```
<script src="https://unpkg.com/react@18/umd/react.development.js"></script>
<script src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"></script>
<script src="https://unpkg.com/@babel/standalone/babel.min.js"></script>
```

This pulls scripts from unpkg.com, a free script distribution service.

Icon by HideMaru for Noun Project



Created by HideMaru
from Noun Project

How to get started?

- Now, below all of the above, add:

```
        <script type="text/babel">
            const rootNode = document.getElementById('root');
            const root = ReactDOM.createRoot(rootNode); // This initializes the application

            // this is JSX code; it will be transpiled by Babel
            // transpiled code is standard JavaScript code that browsers can understand
            root.render(
                <h1>Hello, React!</h1>
            );
        </script>
```

- Open the page in your browser.

Icon by HideMaru for Noun Project

```html
<html>
    <head>
        <meta charset="UTF-8" />
        <meta name="viewport" content="width=device-width, initial-scale=1.0" />
        <title>React App</title>
    </head>
    <body>
        <div id="root"></div>
        <script src="https://unpkg.com/react@18/umd/react.development.js"></script>
        <script src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"></script>
        <script src="https://unpkg.com/@babel/standalone/babel.min.js"></script>
        <!-- type="text/jsx" also works -->
        <script type="text/babel">
            const rootNode = document.getElementById('root');
            const root = ReactDOM.createRoot(rootNode); // This initializes the application

            // this is JSX code; it will be transpiled by Babel
            // transpiled code is standard JavaScript code that browsers can understand
            root.render(
                <h1>Hello, React!</h1>
            );
        </script>
    </body>
</html>
```

React App

File C:/Users/fa_lo/Dropbox/clases/INFO%206150%20Web%20Design%20and%20User%20Experie...

Hello, React!

Elements Console Recorder Sources Network Performance Memory Application

```
<html>
  <head> ... </head>
  <body>
    <div id="root">
      <h1>Hello, React!</h1>
    </div>
    <script src="https://unpkg.com/react@18/umd/react.development.js"></script>
    <script src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"></script>
    <script src="https://unpkg.com/@babel/standalone/babel.min.js"></script>
    <!-- type="text/jsx" also works -->
    <script type="text/babel"> == $0
                const rootNode = document.getElementById('root');
                const root = ReactDOM.createRoot(rootNode); // This initializes the
      application

                // this is JSX code; it will be transpiled by Babel
                // transpiled code is standard JavaScript code that browsers can
      understand
                root.render(
                    <h1>Hello, React!</h1>
                );
    </script>
  </body>
</html>
```

html body script

Styles Computed Layout

margin 0
border 0
padding 0
0 0 0 auto × auto 0 0 0
0
0
0

Filter Show all Gro...

accent-color
auto
align-content
normal
align-items
normal
align-self
auto
alignment-baseline
auto
all

anchor-name
none

What happened?

- The React and React-DOM packages are loaded first.
  - React-DOM allows React to manipulate the DOM.
- Then, we load Babel.js so that we can write JSX.
  - JSX is Javascript, but looks like HTML and compile to HTML.
  - JSX was created to be an easier way to interact with the DOM.
  - Babel is a “transpiler”: a program that converts one language into another.
- In our script, we select an HTML element and initialize our React application to write inside of it.

```
const rootNode = document.getElementById('root');
const root = ReactDOM.createRoot(rootNode); // This initializes the application
```

- Lastly, we output JSX within that node, using the render() method of the React app.

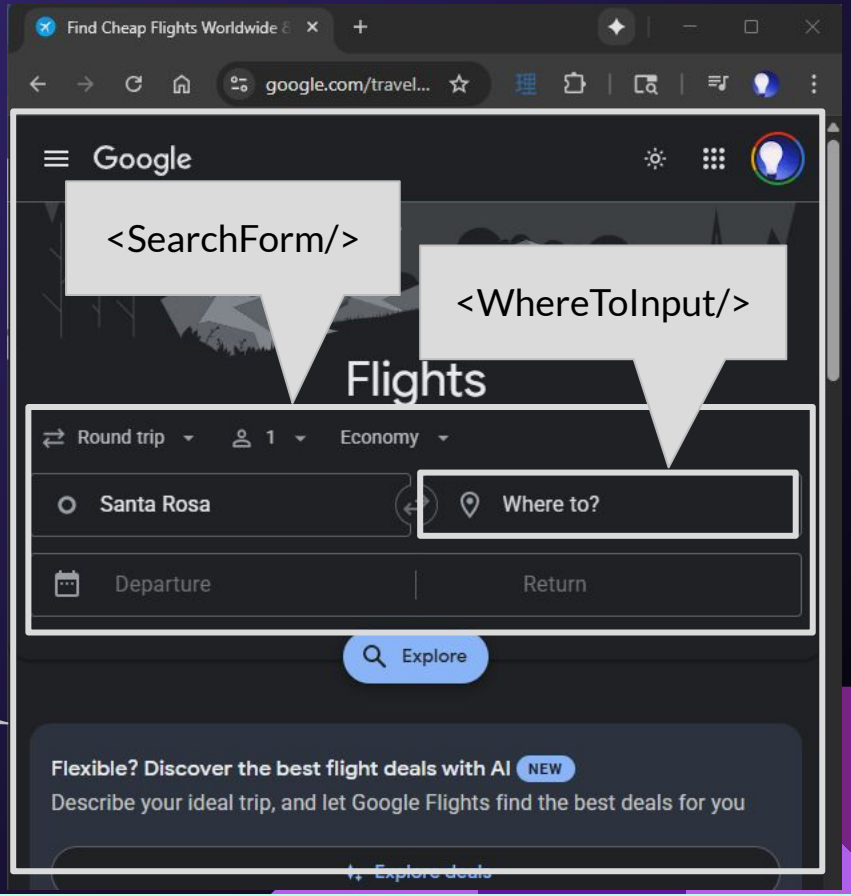
```
root.render(<h1>Hello, React!</h1>);
```

What happened?

- JSX is almost like HTML, but it follows some rules:
  - Use camelCase instead of snake-case (save for a few exceptions)
  - className instead of class
  - All elements (tags) must have a single root. We'll explore this later

Components

A component is a part of your UI. The idea for React and other packages is that you divide your app in components, and each one has different responsibilities or represents something different.



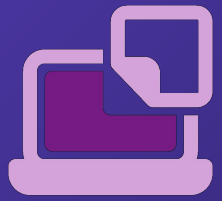
<App/>

Components

React components are just functions that return JSX.

Note: React components can also be Javascript classes, but this is no longer recommended.

The JSX is said to be rendered. (This means it will be shown in the screen.)



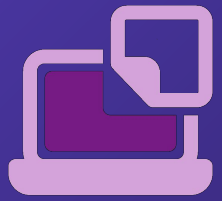
Created by HideMaru
from Noun Project

Making a component

- Change the contents of your <script type="text/babel"> as follows:

```
const rootNode = document.getElementById('root');
const root = ReactDOM.createRoot(rootNode); // This initializes the application

const Title = () => {
    return <h1>Hello, React!</h1>;
};
const Body = () => {
    return <p>This is a simple React application.</p>;
};

// this is JSX code; it will be transpiled by Babel
// transpiled code is standard JavaScript code that browsers can understand
root.render(
    <>
        <Title />
        <Body />
    </>,
);
```

Icon by HideMaru for Noun Project



Created by HideMaru
from Noun Project

Making a component

- Change the contents of your <script type="text/babel"> as follows:

```
const rootNode = document.getElementById('root');
const root = ReactDOM.createRoot(rootNode); // This initializes the application

const Title = () => {
    return <h1>Hello, React!</h1>;
};
const Body = () => {
    return <p>This is a simple React application.</p>;
};

// this is JSX code; it will be transpiled by Babel
// transpiled code is standard JavaScript code that
root.render(
    <>
        <Title />
        <Body />
    </>,
);
```

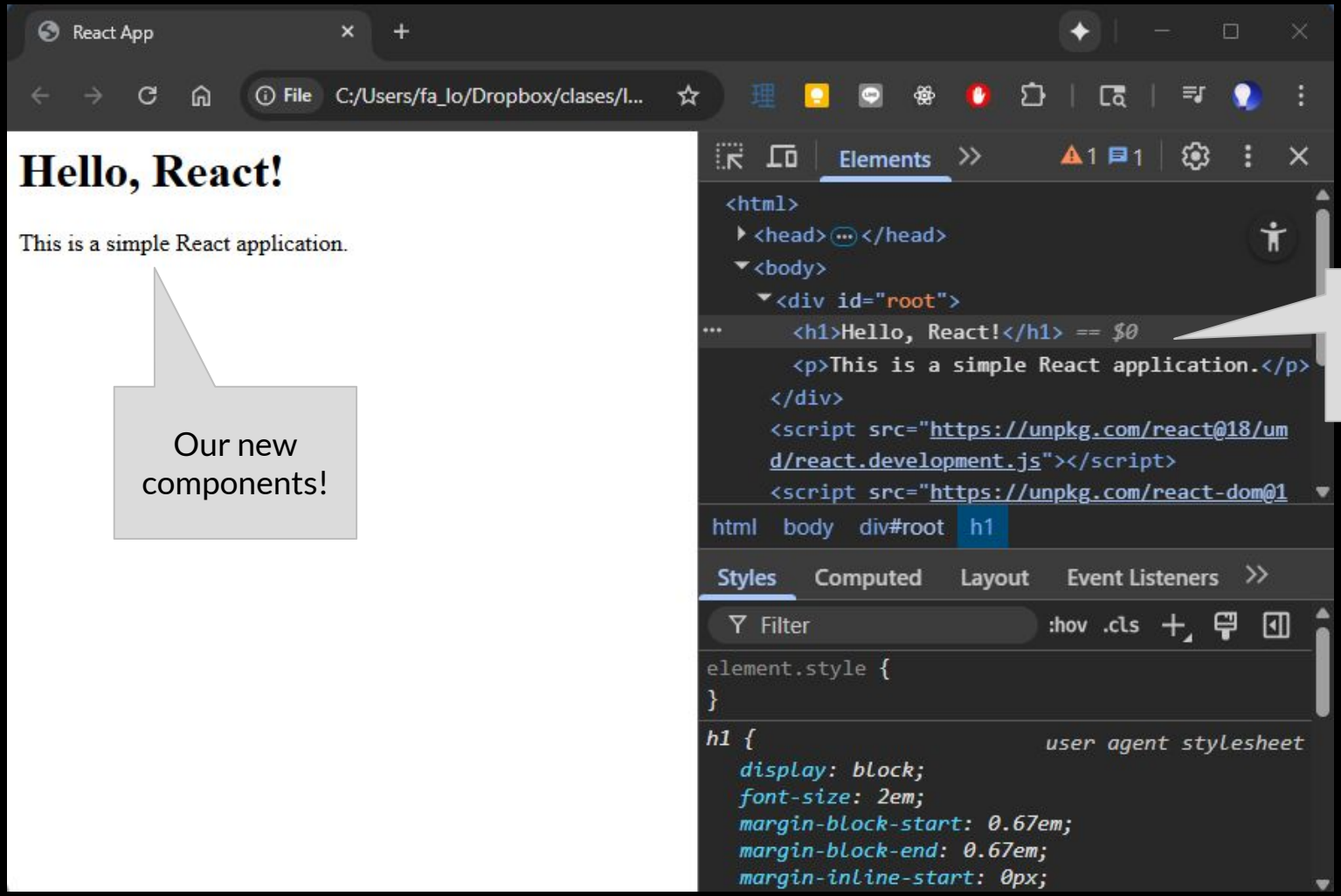
In JSX, if we need to output more than one tag at a time, we use <></> as the parent.

Icon by HideMaru for Noun Project

Components: Rendering more than one node

Remember: While only one root node is permitted, that can have as many children as needed.

```
    return (
        <>
            <h1 style={{ color: props.textColor }}>
                Welcome to {name}
            </h1>
            <h2>Thank you for visiting.</h2>
        </>
    );
```


React App
File
C:/Users/fa_lo/Dropbox/clases/l...
Hello, React!
This is a simple React application.
Our new components!
Elements
<html>
<head> ... </head>
<body>

<h1>Hello, React!</h1> == $0
This is a simple React application.

<script src="https://unpkg.com/react@18/umd/react.development.js"></script>
<script src="https://unpkg.com/react-dom@1
html
body
div#root
h1
Styles
Computed
Layout
Event Listeners
Filter
:hov
.cls
element.style {
}
h1 {
user agent stylesheet
display: block;
font-size: 2em;
margin-block-start: 0.67em;
margin-block-end: 0.67em;
margin-inline-start: 0px;
...get rendered as HTML

Components: Rendering data

To render data, wrap it in brackets (`{}`).

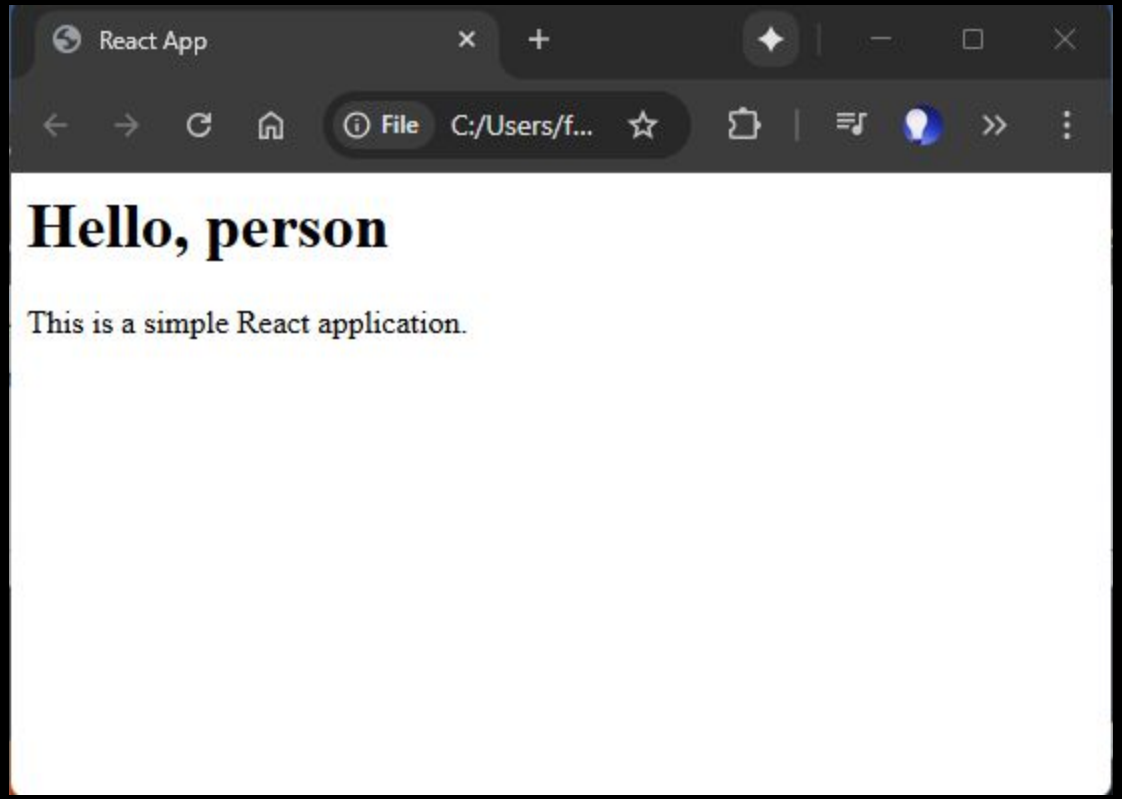
Let's define a constant within the Title component and render (output) it.

- Change your code to look like this:

```
const Title = () => {
    const name = "person";
    return <h1>Hello, {name}</h1>;
};
```

Note: Pure variables, values, and arrays can be rendered this way, but NOT objects.

- If you really need to output an object as is, you can use JSON.stringify() to convert it to a string first.


React App
File
C:/Users/f...
Hello, person
This is a simple React application.

Components: Passing props

In React, the parameters passed to a component are called “props”. Props are always received in an object.

Let’s add props to our Title component:

```
const Title = (props) => {
    const name = "person";
    console.log(props);
    return <h1 style={{ color: props.textColor }}>Hello, {name}</h1>;
};
```

- We are adding a prop and then rendering it.

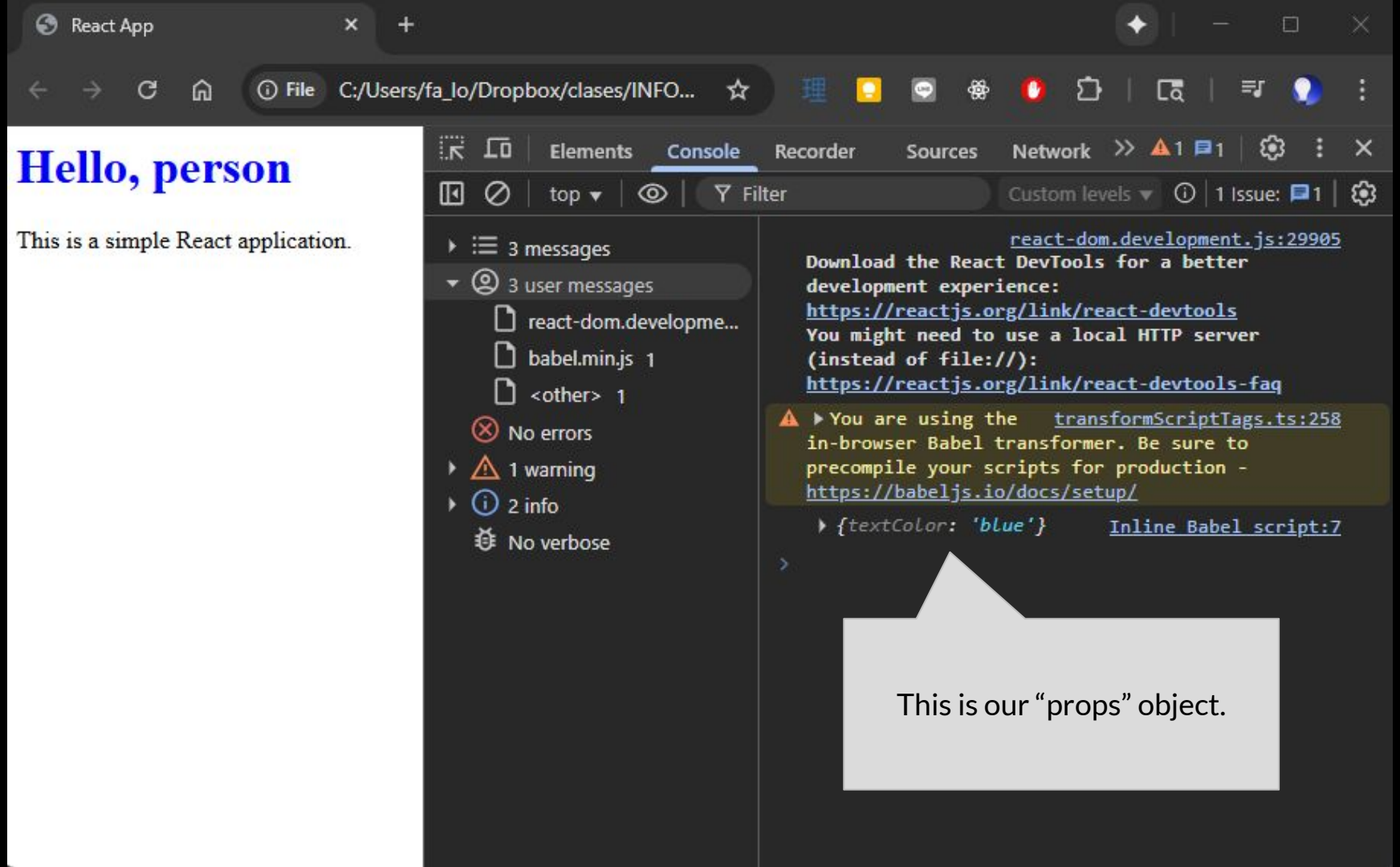
Components: Passing props

Now we have to pass those props when we use `<Title/>`.

Let's change how we call it:

```
root.render(
    <>
        <Title textColor="blue" />
        <Body />
    </>,
);
```

String values are passed like this.


React App
File
C:/Users/fa_lo/Dropbox/clases/INFO...
Hello, person
This is a simple React application.
Elements
Console
Recorder
Sources
Network
top
Filter
Custom levels
1 Issue: 1
3 messages
3 user messages
react-dom.developme...
babel.min.js 1
<other> 1
No errors
1 warning
2 info
No verbose
react-dom.development.js:29905
Download the React DevTools for a better development experience:
https://reactjs.org/link/react-devtools
You might need to use a local HTTP server (instead of file://):
https://reactjs.org/link/react-devtools-faq
You are using the in-browser Babel transformer. Be sure to precompile your scripts for production - https://babeljs.io/docs/setup/
transformScriptTags.ts:258
{textColor: 'blue'}
Inline Babel script:7
This is our “props” object.

Components: Passing props

Note: props are passed in brackets, except when they are string values. Any Javascript value can be passed as prop, including objects.

```
<img src={logo} className="App-logo" alt="logo" />
```

Variable (can be of any type, even string)

String values

Components: Nesting

We can use components inside of other components, creating a component tree (similar to HTML). This means we can organize our code.

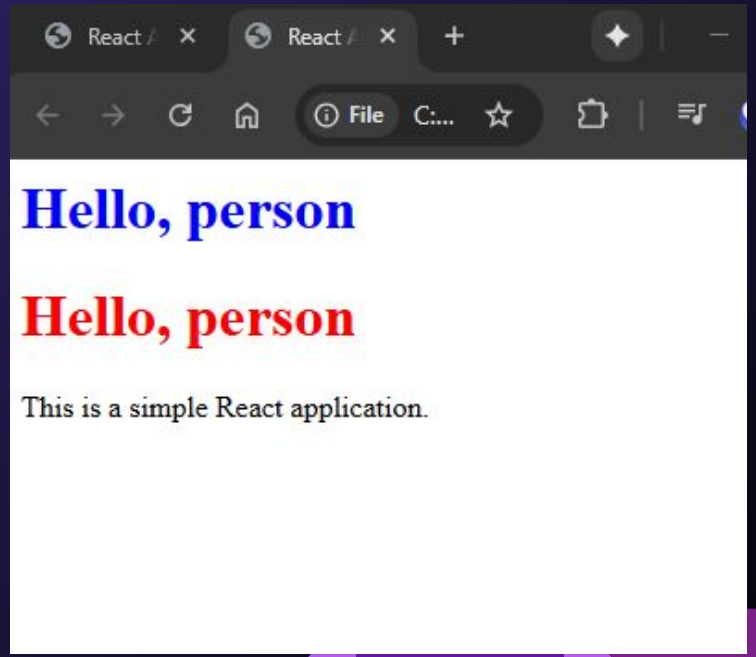
Let's create an App component as the root (this is a good practice). Your code should render the same in the browser.

```
const App = () => {
    return (
        <>
            <Title textColor="blue" />
            <Body />
        </>
    );
};

root.render(
    <App />
);
```

Components: Reusing

Components are generic, so we can reuse them:

```
const App = () => {
    return (
        <>
            <Title textColor="blue" />
            <Title textColor="red" />
            <Body />

        </>
    );
};

root.render(
    <App />
);
```



Components: Prop default values

What happens if the user did not pass a color? Let's add a default value.

First, let's destructure props so that it's easier to use. Instead of

```
const Title = (props) => {
```

we will now use:

```
const Title = ({ textColor }) => {
```

Also remove the `console.log(props)` line.

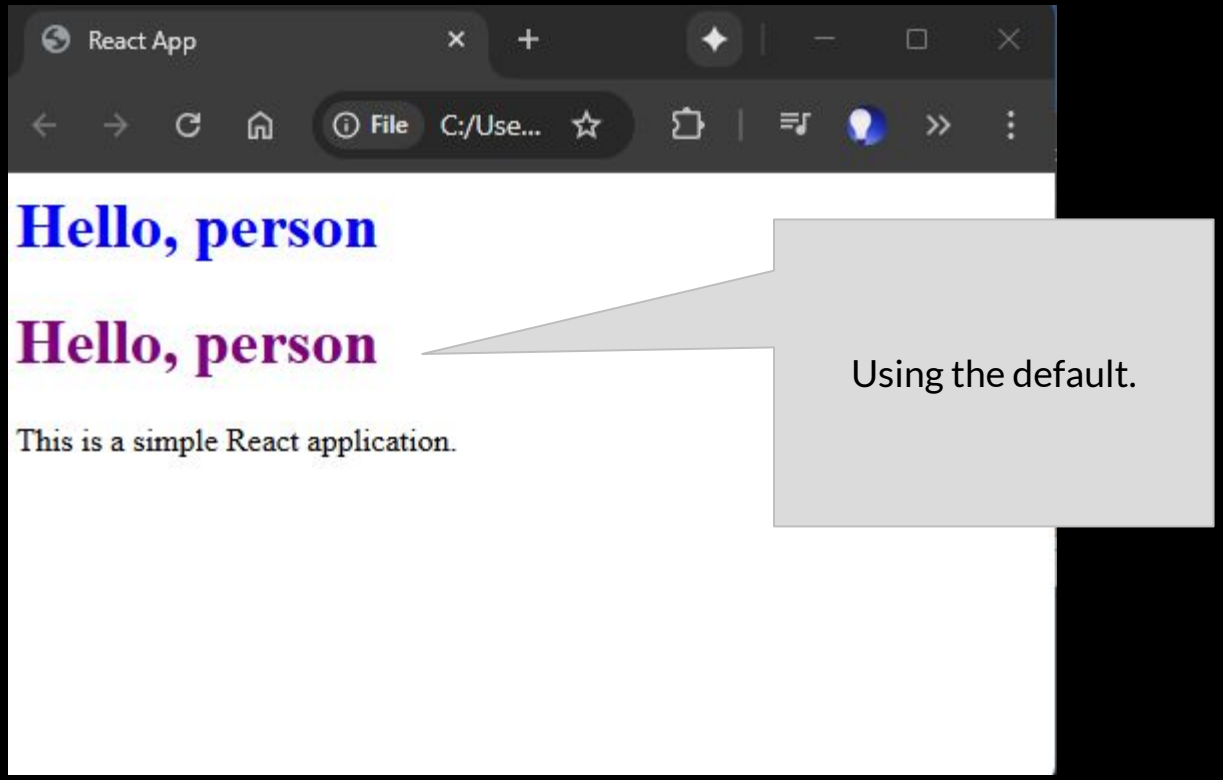
Components: Prop default values

Now let's provide a default value like this:

```
const Title = ({ textColor="purple" }) => {
```

And let's "forget" to pass the prop:

```
const App = () => {
    return (
        <>
            <Title textColor="blue" />
            <Title /> // removed the textColor prop
```


React App
File
C:/Use...
Hello, person
Hello, person
This is a simple React application.
Using the default.

Components: Prop default values with ternary operators

We can also output ternary operators or the result of functions. For example, even if we do not have a default value for `textColor`, we can do this:

```
const Title = ({ textColor }) => {
    const name = "person";
    return <h1 style={{ color: textColor ? textColor : "purple" }}>Hello, {name}</h1>;
};
```

Rendering arrays

Rendering arrays is common in React. There is a neat trick for it: use the array's `map` method.

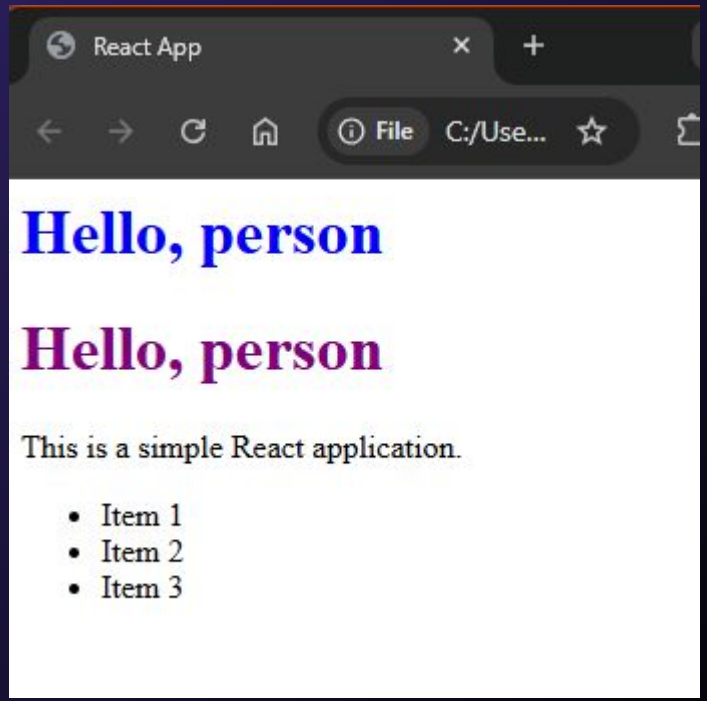
Let's create a new component to demonstrate:

```
const List = () => {
    const items = ["Item 1", "Item 2", "Item 3"];
    return (
        <ul>
            {items.map((item, index) => (
                <li key={index}>{item}</li>
            ))}
        </ul>
    );
};
```

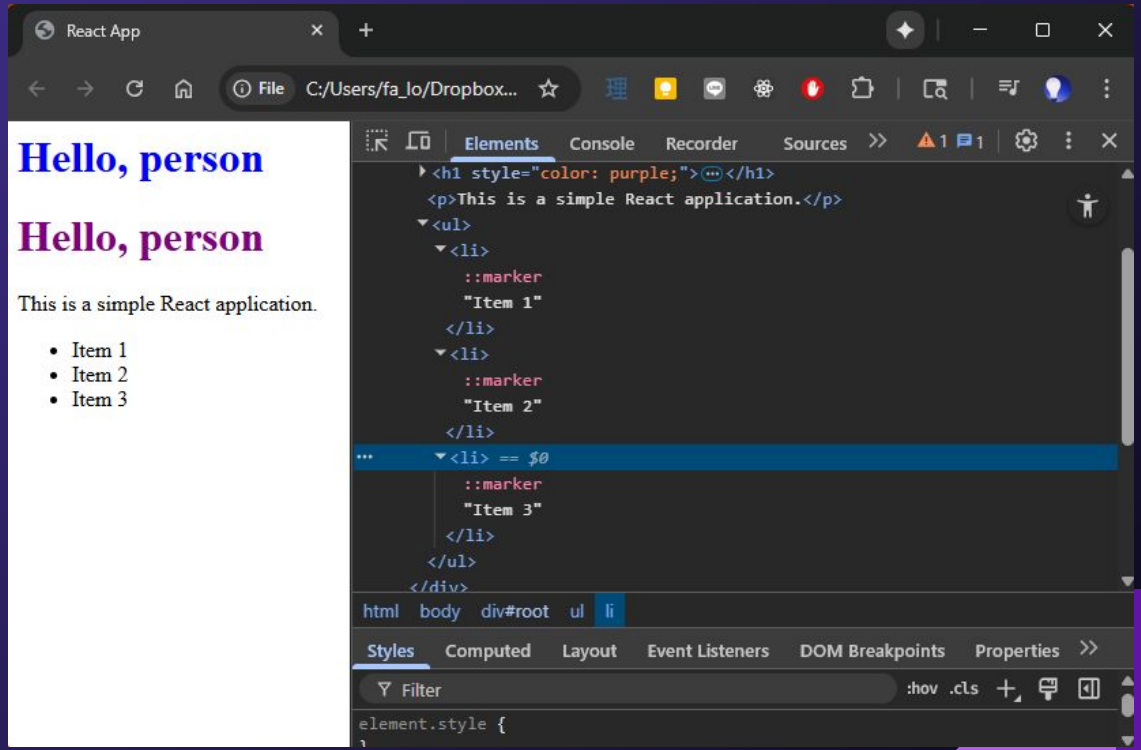
Rendering arrays

And let's use our component:

```
const App = () => {
    return (
        <>
            <Title textColor="blue" />
            <Title />
            <Body />
            <List />
        </>
    );
};
```



Rendering arrays



Components and re-renders

Note: each time any input to a component changes, the whole component will re-render. If the component renders (calls) other components, they will re-render too.

This is supposed to happen! This allows the output to be updated according to each input change. This is similar to how event handlers work.

However, “re-rendering” means that the whole function runs again. If the component makes complex calculations, they will happen again. This can be wasteful and make the application much slower.

Components and re-renders

To avoid unnecessary re-renders:

- Do not make calculations inside the render itself.

```
return (
        <>
            <h1 style={styleObj}>
                Welcome to {name}, you are visitor number {doHeavyCalculation()}.
            </h1>
```

Don't!
Do this outside of the return() instead

- Use React features such as hooks to reuse values. (We will see this next.)

Structure of a component (so far)

```
// parameters are called "props"
function SomeComponent(props) {
    // state; define your constants / variables

    // logic, operations; calculations and event handlers can go here

    // render; only one root element is allowed
    return (
        <>some JSX</>
    );
}
```

Hooks

Hooks

Hooks are special functions provided by React. They allow access to special React features.

Hooks have certain rules. The most important is:

- They can only be called from the top level of a React component.
  - This means they cannot be called from within an if {} block or any cycle block (for, while, do…while). This is because a component must always render the same number of hooks.

Thanks to these rules, hooks are highly *performant*, which means apps can do advanced stuff very quickly.

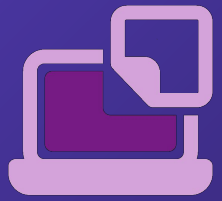
It is also possible for you to create your own hooks. Hooks can also call other hooks.

The useState hook

The state hook is possibly the most useful of all hooks.

Data in a “normal” variable in the function gets recalculated each time it renders. As already stated, this is wasteful.

The state hook lets the component “remember” the value of the variable across re-renders.



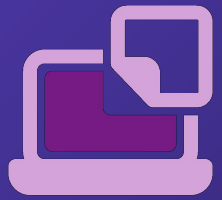
Created by HideMaru
from Noun Project

The useState hook

Let’s create a simple click counter component. It will render a button and the number of times we’ve clicked on the button:

```
        <script type="text/babel">
            const rootNode = document.getElementById('root');
            const root = ReactDOM.createRoot(rootNode); // This initializes the application
            const ClickCounter = () =>{
                let counter = 0;
                const clickHandler = () => {
                    counter += 1;
                };

                return (
                    <div>
                        <p>Clicks: {counter}</p>
                        <button onClick={clickHandler}>Click me</button>
                    </div>
                );
            }
```

Icon by HideMaru for Noun Project



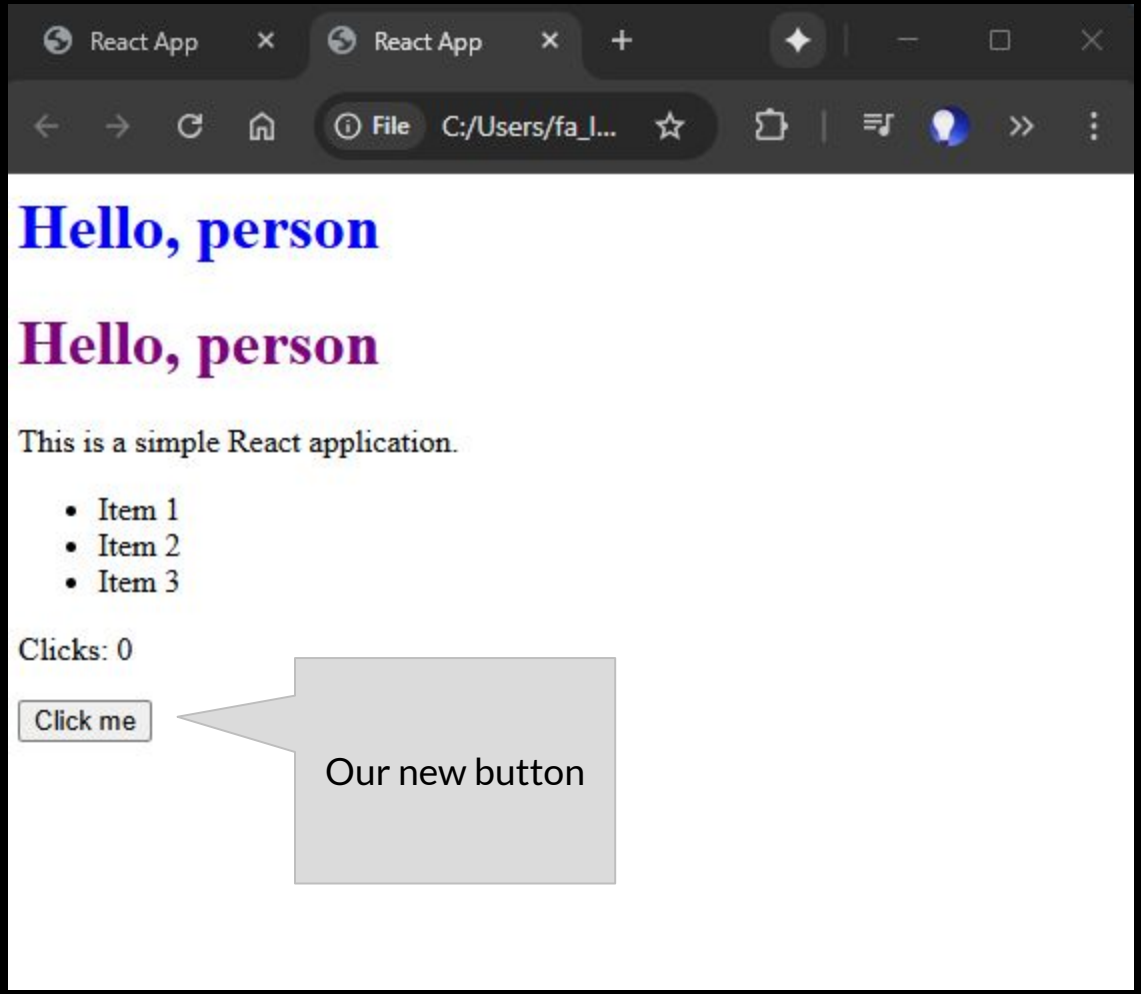
Created by HideMaru
from Noun Project

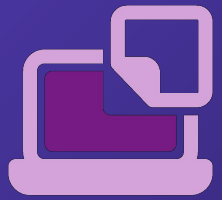
The useState hook

Let’s render it:

```
const App = () => {
    return (
        <>
            <Title textColor="blue" />
            <Title />
            <Body />
            <List />
            <ClickCounter />
        </>
    );
};
```

Icon by HideMaru for Noun Project


React App
React App
File
C:/Users/fa_l...
Hello, person
Hello, person
This is a simple React application.
Item 1
Item 2
Item 3
Clicks: 0
Click me
Our new button



Created by HideMaru
from Noun Project

The useState hook

It should… not work.

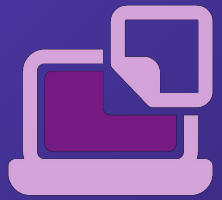
Why does this not work?

- Every time we click on the button, the component re-renders - since it just received user input.
- When the component re-renders, the variable counter gets re-declared and its value reset to 0.

```
const ClickCounter = () => {
    let counter = 0; // this runs again!
```

What we need is to preserve its value between re-renders: we need this to be state.

Icon by HideMaru for Noun Project



Created by HideMaru
from Noun Project

The useState hook

Change `ClickCounter` to look like this:

```
const ClickCounter = () =>{
    const [counter, setCounter] = React.useState(0);
    const clickHandler = () => {
        setCounter((oldValue) => {
            return oldValue + 1;
        });
    };
```

Test it again. It should work!

Icon by HideMaru for Noun Project

The useState hook

How to use it:

```
const [counter, setCounter] = useState(0);
```

- The parameter to `useState()` will be the initial value for your state; in this case, 0.
- It returns an array with two variables:
  - The first variable contains the state value. Read from it.
    - Don't write to this variable: it will not work. (Don't do `counter = counter + 1`).
  - The second is a function: call it to write a new value to the state.
    - The parameter of this function is another function (a callback).
      - The callback gets the old value of the state and returns the new value.
    - Alternatively, instead of a callback, you can just return the new value:

      ```
      setCounter(counter + 1);
      ```

      However, in some cases this will not work.

Ideally, only pure values are state (eg. not objects, not arrays) to preserve efficiency.

An aside: immutability

In Javascript it's possible to change the values contained by arrays and objects, even if the arrays and objects themselves

React is based on a functional programming paradigm. One of the main concepts of it is immutability.

The immutability concept says that once created, the value of a variable should not change. In other words, we should only use constants.

In React (and in Javascript in general) it's a good idea to use immutable objects and arrays.

- This means that if we need to change an object, we create a copy of it with the changed value instead. Same for arrays.
- This allows us to make sure React understands that the object or array has changed, so the application is refreshed.

An aside: immutability

So, instead of:

```
const anObj = { a: 1, b: 2, c: 3 };
anObj.a = 4;
```

You should do:

```
let newObj = {...anObj, a: 4};
```

This is what we will be doing when changing objects in state.

An aside: Pure functions

React is built on the idea that its components are (mostly) pure functions.

What is a pure function?

- A pure function will always return the same output if given the same input. It's like a mathematical formula.
  - In other words, a pure function is idempotent. (We studied this concept in the previous lesson.)
  - For example: `function multiply(a, b) => a * b;` is a pure function.
  - If we call `multiply(3, 2)` it will always return 6, no matter what.

By the way: even in pure Javascript, or in any other programming language, pure functions are a good idea! They are simpler to program and test, and mean having much less bugs!

An aside: Pure functions

Pure functions vs. side effects:

A function that is not pure will contain “side effects”.

A side effect is anything that happens outside of the function “minding its own business”. For example:

- Changing the values of variables outside of its own scope
- Starting and stopping other operations
- Etc.

Event handlers can also be considered side effects.

An aside: Pure functions

Pure functions vs. side effects:

React needs that the rendering of the function behaves as a pure function.

For example, this is not good:

```
var a = 0;

const Count = () => {
    a = a + 1;
    return (
        <p>{a}</p>
    )
}

const App = () => {
    return (
        <>
            <Count />
            <Count />
        </>
    )
}
```

An aside: Pure functions

Pure functions vs. side effects:

React needs that the rendering of the function behaves as a pure function.

For example, this is not good:

```
var a = 0;

const Count = () => {
    a = a + 1;
    return (
        <p>{a}</p>
    )
}

const App = () => {
    return (
        <>
            <Count />
            <Count />
        </>
    )
}
```

Passing no props.

Our component is changing and using external state (a variable in the parent).

An aside: Pure functions

Pure functions vs. side effects:

React needs that the rendering of the function behaves as a pure function.

For example, this is not good:

```
var a = 0;

const Count
    a = a +
    return
        <p>
    )
}

const App = () => {
    return (
        <>
            <Count />
            <Count />
        </>
    )
}
```

The same component, called with the same props (none, in this case), is producing different results. React is sad. :(

```
<p>1</p>
<p>2</p>
```

An aside: Pure functions

Pure functions vs. side effects:

This is one way of fixing it: pass the number to render as a prop.

```
const Count = (props) => {
    return (
        <p>{props.a}</p>
    )
}

const App = () => {
    return (
        <>
            <Count a={1} />
            <Count a={2} />
        </>
    )
}
```

```
<p>1</p>
<p>2</p>
```

An aside: Pure functions

Pure functions vs. side effects:

This is the same, but more efficient. You may find code like this in the internet.

```
const Count = (props) => {
    return (
        <p>{props.a}</p>
    )
}

const App = () => {
    const countArr = [];
    for (let a = 1; a <= 2; a++) {
        countArr.push(<Count a={a} />);
    }
    return (
        <>
            {countArr}
        </>
    )
}
```

```
<p>1</p>
<p>2</p>
```

An aside: “Good” side effects

Side effects change the program’s state: that is, the data outside of the component.

We do want some side effects, such as event handlers. We need them for any user input, operations with the servers, etc.

An aside: “Good” side effects

But we must be careful with them, because React renders components when it wants, meaning we can lose control on how the effect happens.

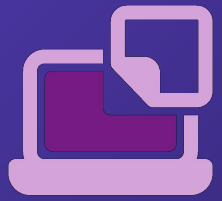
We will see an example in the following slides.

Handling side effects in React: the useEffect hook

In addition to events (which come standard with Javascript), React includes a new system: effects.

React's useEffect hook makes it easier to "synchronize" (communicate) with systems outside of React itself. For example:

- Loading and receiving data from a server
- Writing data to a file, or reading from a file
- Control non-React components



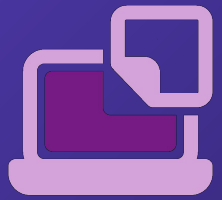
Created by HideMaru
from Noun Project

Handling side effects in React: the useEffect hook

Example: Let’s update our click counter.

- When clicking the button, it will count how many times it’s been clicked.
- The number of times will display in two places: the button text and the <title></title> of the page.
  - <title> is the text that shows on the name of the window tab!

Icon by HideMaru for Noun Project



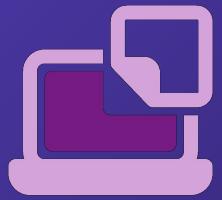
Created by HideMaru
from Noun Project

Handling side effects in React: the useEffect hook

Change `ClickCounter` again:

```
const ClickCounter = () =>{
    const [counter, setCounter] = React.useState(0);
    const clickHandler = () => {
        setCounter((oldValue) => {
            return oldValue + 1;
        });
        document.title = `Clicked ${counter} times`;
    };
```

Icon by HideMaru for Noun Project

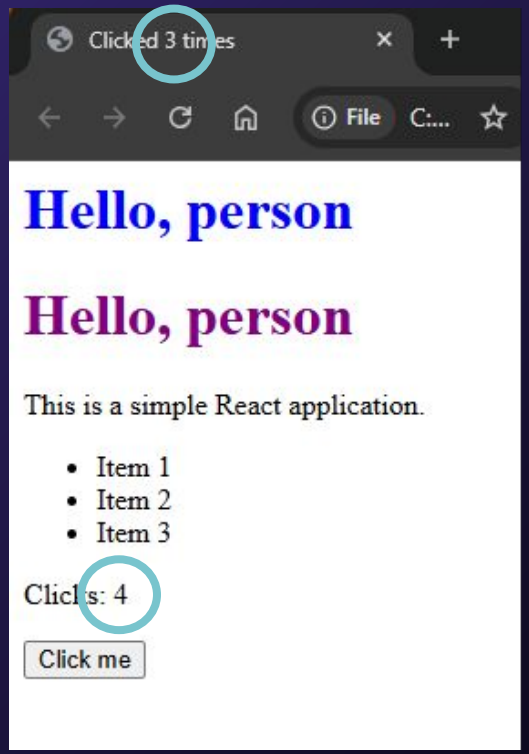


Created by HideMaru
from Noun Project

Handling side effects in React: the useEffect hook

It *almost* works.

- The button always shows a number higher than the title! Why?

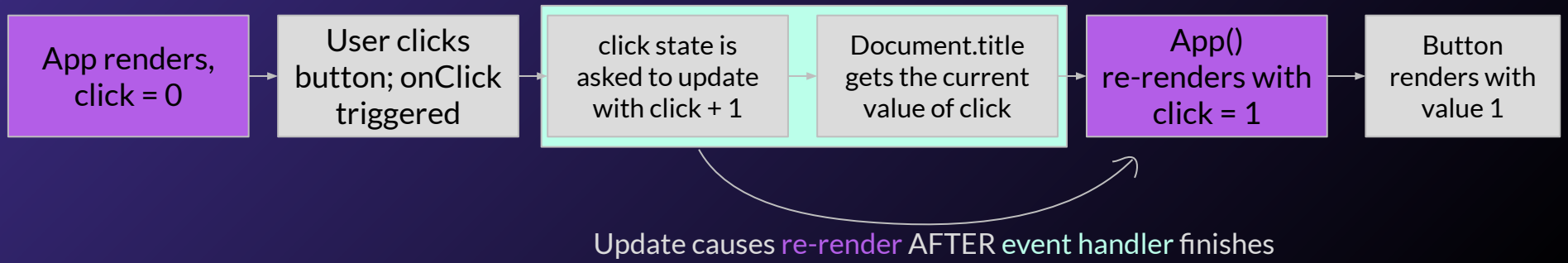


Icon by HideMaru for Noun Project

Handling side effects in React: the useEffect hook

To understand this, keep in mind the following:

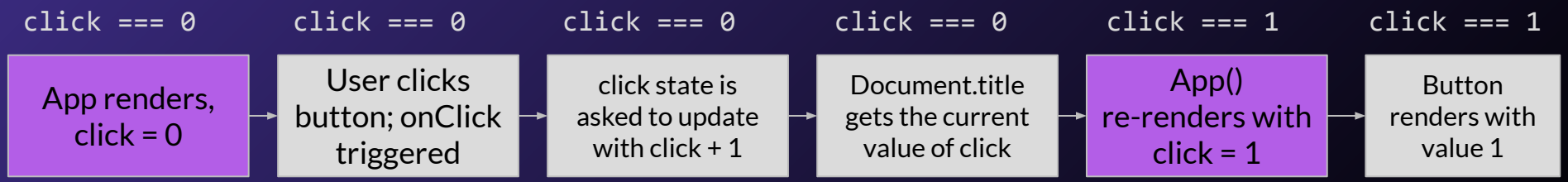
- When the props or the state of the component change, the component re-renders… after they change.
  - This allows the components to *react* to the change, showing the latest version of the data.
    - (yes, the name comes from this!)
- When setCount is called, this causes the app to re-render after the event handler finishes.
  - After re-rendering, the button (in React) shows the value “1”



Handling side effects in React: the useEffect hook

- BUT the state actually only gets the new value after re-rendering!

When the title gets updated, it gets updated with “0”.



Handling side effects in React: the useEffect hook

We need to update sooner. What if we update as soon as we enter the component?

```
const ClickCounter = () =>{
    const [counter, setCounter] = React.useState(0);
    document.title = `Clicked ${counter} times`;
    const clickHandler = () => {
        setCounter((oldValue) => {
            return oldValue + 1;
        });
    };
```

Handling side effects in React: the useEffect hook

This works, but… we've created an ugly side effect! This may affect rendering.

```
const ClickCounter = () =>{
    const [counter, setCounter] = React.useState(0);
    document.title = `Clicked ${counter} times`;
    const clickHandler = () => {
        setCounter((oldValue) => {
            return oldValue + 1;
        });
    };
```

Affects something that is not the return value (JSX): document.title is outside of React

Remember: Code in the main function should generate the output (render) out of the inputs (props and state). Everything else is a side effect. This is no longer a pure function.

Handling side effects in React: the useEffect hook

What if there was a prop and the prop changed as well?

- This would trigger another re-render.
- This means `document.title = `Clicked ${count} times`;` gets executed again.

We have lost control over the side effect. If this was a destructive update (such as summing a number or deleting something), we would be in trouble.

We need more control.

```
const ClickCounter = ({ user }) =>{
    const [counter, setCounter] =
React.useState(0);
    document.title = `Clicked ${counter} times`;
```

```
...
<ClickCounter user={user} />
```

(suppose `user` changes, for example, due to logging in)

Handling side effects in React: the useEffect hook

Let’s wrap the side effect in a useEffect hook:

```
const ClickCounter = ({ user }) =>{
    const [counter, setCounter] = React.useState(0);

    React.useEffect(() => {
        document.title = `Clicked ${counter} times`;
    }, [counter]); // Update the document title whenever counter changes

    const clickHandler = () => {
        setCounter((oldValue) => {
            return oldValue + 1;
        });
    };
```

Handling side effects in React: the useEffect hook

Let's wrap the side effect in a useEffect hook:

```
const ClickCounter = ({ user }) =>{
    const [counter, setCounter] = React.useState(0);

    React.useEffect(() => {
        document.title = `Clicked ${counter} times`;
    }, [counter]); // Update the document title whenever counter changes

    const clickHandler = () => {
        setCounter((oldValue) => {
            return oldValue + 1;
        });
    };
```

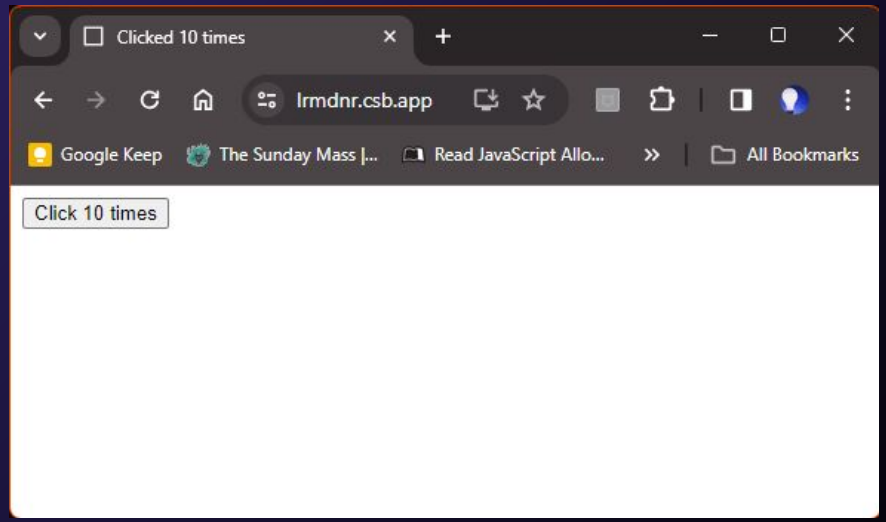
The first parameter of useEffect is a callback. We put our side effect within it.

Handling side effects in React: the useEffect hook

It still works!

Now, to be fair, it's still not very different to not having the hook. We said we wanted control.

Here is where we can improve things. We can tell React when to run the hook (and when not to run it), instead of running it on each render.



Handling side effects in React: the useEffect hook

The second parameter to useEffect is an array.

This is an optional parameter; you don't need to put it. If it's not there, the effect will always run (every time the component renders).

If the array is there, the effect only runs when the variables listed in the array change.

If you use any variable or prop inside the effect, you should put them in the array.

```
React.useEffect(() => {
  document.title = `Clicked ${count} times`;
}, [count]);
```

The effect only runs if count changes.

Now we are under control! The effect only runs when we want it to run - even if the component re-renders more often.

Don’t update internal state in an effect

IMPORTANT!

- Events run as a result of rendering. They happen after the component renders.
- Changing state re-renders.

Therefore, if your useEffect changes state, this will cause an endless loop!

```
const App = () => {
    const [count, setCount] =
React.useState(0);

    React.useEffect(() => {
        setCount((oldCount) => oldCount +
1); // WILL CRASH YOUR BROWSER!!!
    });

    return (
        <p>{count}</p>
    );
}
```

Don’t update internal state in an effect

If you need to do this, you need to add a condition to your effect, such as:

```
const App = () => {
    const [count, setCount] = React.useState(0);

    React.useEffect(() => {
        if (count < 2) {
            setCount((oldCount) => oldCount + 1);
        }
    });

    return (
        <p>{count}</p>
    );
}
```

In any case, you **should not** create an effect just to update internal state. You don’t need it!

Handling side effects in React: the useEffect hook

The difference between an event and an effect is that:

- An event happens when something happens on the program, such as the user typing, the page loading, a connection opening…
- An effect (in theory) happens always, that is, every time a component renders.
  - As we saw before, we can actually control when it runs and when it doesn't…
  - …but it will always run because the component rendered.

Next.js

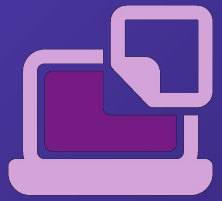
React Frameworks

The “real” way to create a React app usually is through a framework. This is what the React developers recommend.

Full-stack frameworks generally use node.js both to compile your app during development and to create a production-optimized app. They also take care of many additional concerns, such as routing.

- In this context, routing means deciding what root component will be used when the user opens a URL such as yourapp.com/users or yourapp.com/profile.

Out of the many frameworks out there we will study Next.js because it’s popular and well maintained. Other alternatives include React Router and Expo (for native apps).

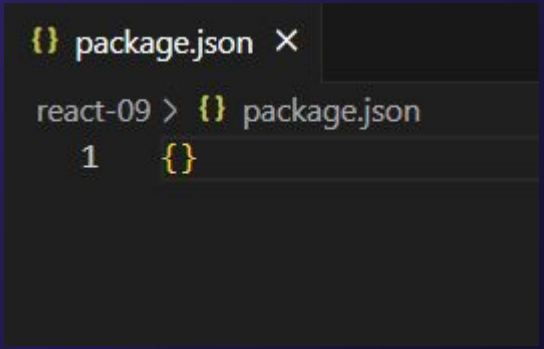


Created by HideMaru
from Noun Project

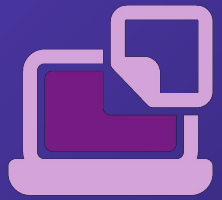
Converting the app to Next.js

Let's convert our current application into a Next.js application. We will roughly follow the same steps as the Next tutorial.

- First, create a new file called "package.json" in your directory. Save in it an empty object: `{}`



Icon by HideMaru for Noun Project



Created by HideMaru
from Noun Project

Converting the app to Next.js

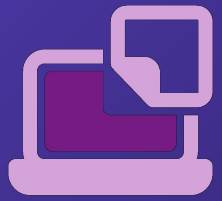
- Then, in that same directory, open your console or terminal and run

```
npm install react@latest react-dom@latest next@latest
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell - react-09

```
PS C:\Users\fa_lo\Dropbox\clases\INFO 6150 Web Design and User Experience Engineering\2025-03 (fall)\assets\14 React Fundamentals\react-09> npm install r
eact@latest react-dom@latest next@latest
>>

added 20 packages, and audited 21 packages in 30s

5 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\Users\fa_lo\Dropbox\clases\INFO 6150 Web Design and User Experience Engineering\2025-03 (fall)\assets\14 React Fundamentals\react-09>
```

Icon by HideMaru for Noun Project



Created by HideMaru
from Noun Project

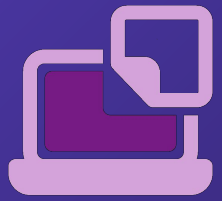
Converting the app to Next.js

The command just updated your `package.json` file. It should look more or less like this:

```
{
  "dependencies": {
    "next": "^16.0.1",
    "react": "^19.2.0",
    "react-dom": "^19.2.0"
  }
}
```

You should now also have a `package-lock.json` and a `node_modules` directory.

Icon by HideMaru for Noun Project



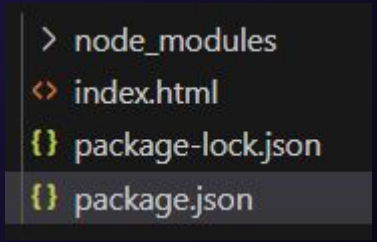
Created by HideMaru
from Noun Project

Converting the app to Next.js

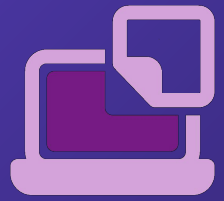
The command just updated your `package.json` file. It should look more or less like this:

```
{
  "dependencies": {
    "next": "^16.0.1",
    "react": "^19.2.0",
    "react-dom": "^19.2.0"
  }
}
```

You should now also have a `package-lock.json` and a `node_modules` directory.



Icon by HideMaru for Noun Project



Created by HideMaru
from Noun Project

Making a JSX file

Next, let's convert your index.html into a JSX file:

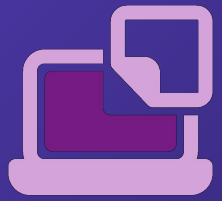
- Delete all of the code outside of `<script type="text/babel">`, including the <script> tag. Only Javascript should remain.
- Delete the two lines `const rootNode = document.getElementById('root');` and `const root = ReactDOM.createRoot(rootNode);`
- Delete

```
root.render(
    <App />
);
```

You should be left with five functions only: `ClickCounter`, `Title`, `Body`, `List`, and `App`.

Rename the file to `index.jsx`.

Icon by HideMaru for Noun Project

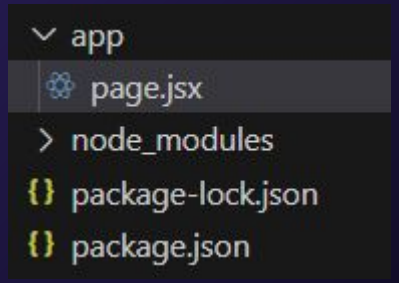


Created by HideMaru
from Noun Project

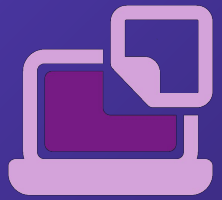
The routing system

Now we'll convert this file to a page within Next's routing system.

- Create a new directory named `app`. Move your file to `app` and change its name to `page.jsx`.



Icon by HideMaru for Noun Project



Created by HideMaru
from Noun Project

The routing system

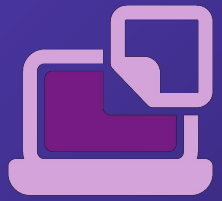
Now we'll convert this file to a page within Next's routing system.

- Create a new directory named `app`. Move your file to `app` and change its name to `page.jsx`.
- In page.jsx, rename your `App` component to `HomePage`.
- At the end of the file, add this line:

```
export default HomePage;
```

which will identify what component is the "main" one within the page.

```
};
const HomePage = () => {
    return (
        <>
            <Title textColor="blue" />
            <Title />
            <Body />
            <List />
            <ClickCounter />
        </>
    );
};

export default HomePage;
```



Created by HideMaru
from Noun Project

Adding the dev server script

Next.js contains a HTTP server that you can run to see page changes while you develop.

Add the following to package.json:

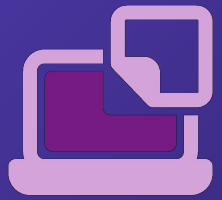
```
  "scripts": {
    "dev": "next dev"
  },
```

No comma here!

Don't forget this comma!

```
{
  ▷ Debug
  "scripts": {
    "dev": "next dev"
  },
  "dependencies": {
    "next": "^16.0.1",
    "react": "^19.2.0",
    "react-dom": "^19.2.0"
  }
}
```

Icon by HideMaru for Noun Project



Created by HideMaru
from Noun Project

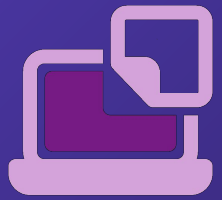
Adding the dev server script

Now, in your terminal, run `npm run dev`. This will attempt to compile the pages and start the server.

```
PS C:\Users\fa_lo\Dropbox\clases\INFO 6150 Web Design and User Experience Engineering\2025-03 (fall)\assets\14 React Fundamentals\react-09> npm run dev
   See https://nextjs.org/docs/app/api-reference/config/next-config-js/turbopack#root-directory for more information.
 Detected additional lockfiles:
   * C:\Users\fa_lo\Dropbox\clases\INFO 6150 Web Design and User Experience Engineering\2025-03 (fall)\assets\14 React Fundamentals\react-09\package-lock.
json

   ▲ Next.js 16.0.1 (Turbopack)
   - Local:        http://localhost:3000
   - Network:      http://10.5.0.2:3000

 ✓ Starting...
 ✓ Ready in 1532ms
```

After this, open your browser to `http://localhost:3000`.

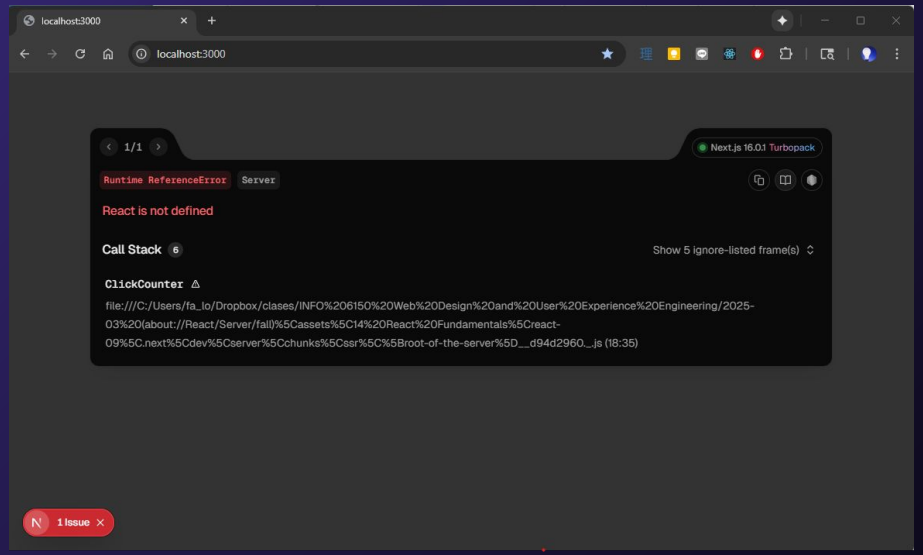
Icon by HideMaru for Noun Project



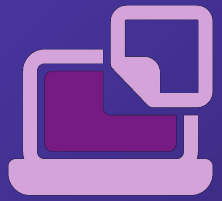
Created by HideMaru
from Noun Project

Adding the dev server script

You should see some errors in the page. The error display will tell you where the error is. This only appears during development.



Icon by HideMaru for Noun Project



Created by HideMaru
from Noun Project

Importing modules

The error tells us “React” does not exist. Let’s add it back to the file, but now we’ll use the JavaScript module system that any node.js project can use.

Add the following at the top of `page.jsx`:

```
import React from "react";
```

Your browser should refresh itself and show another error.

localhost:3000

localhost:3000

1/1

Next.js 16.0.1 Turbopack

Runtime TypeError Server

useState only works in Client Components. Add the "use client" directive at the top of the file to use it. Read more: https://nextjs.org/docs/messages/react-client-hook-in-server-component

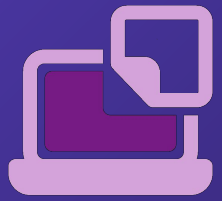
Call Stack 6

Show 5 ignore-listed frame(s)

ClickCounter

file:///C:/Users/fa_lo/Dropbox/clases/INFO%206150%20Web%20Design%20and%20User%20Experience%20Engineering/2025-03%20(about://React/Server/fall)%5Cassets%5C14%20React%20Fundamentals%5Creact-09%5C.next%5Cdev%5Cserver%5Cchunks%5Cssr%5C%5Broot-of-the-server%5D__d94d2960._.js (20:406)

1 Issue



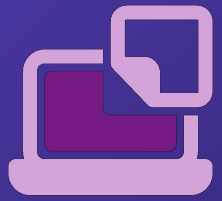
Created by HideMaru
from Noun Project

React Server Components

This time, the error tells us we cannot use the `useState` hook because the application is trying to render in the backend.

- By default, Next applications use React Server Components, a form of server-side rendering for React.
  - The server precalculates and renders React, then sends the results to the client.
  - Apps can be faster when being shown for the first time.
- However, useState and other client-only functions cannot be used on React Server Components. We need to mark them so that they are only run in the client.

Icon by HideMaru for Noun Project



Created by HideMaru
from Noun Project

React Server Components

Let's fix the error by separating and marking the client-only components:

- Inside /app, create a new file: `ClickCounter.jsx`
- Cut your ClickCounter component from page.jsx and paste it to the new file.
- Add `export default ClickCounter;` at the end of the file.
- At the top of the file, add:

```
"use client";
import React from "react";
```

The "`use client`" tells React this file should only be compiled in the client (the browser).

Icon by HideMaru for Noun Project

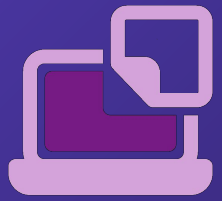
EXPLORER

- 14 REACT FUNDAMENTALS
  - react-01
  - react-02
  - react-03
  - react-04
  - react-05
  - react-06
  - react-07
  - react-08
  - react-09
    - .next
    - app
      - ClickCounter.jsx
      - page.jsx
    - node_modules
    - package-lock.json
    - package.json
  - react-custom
  - index.html

page.jsx | ClickCounter.jsx | package.json

react-09 > app > ClickCounter.jsx > ClickCounter

```jsx
"use client";
import React from "react";

const ClickCounter = () =>{
    const [counter, setCounter] = React.useState(0);

    React.useEffect(() => {
        document.title = `Clicked ${counter} times`;
    }, [counter]); // Update the document title whenever counter changes

    const clickHandler = () => {
        setCounter((oldValue) => {
            return oldValue + 1;
        });
    };

    return (
        <div>
            <p>Clicks: {counter}</p>
            <button onClick={clickHandler}>Click me</button>
        </div>
    );
}
export default ClickCounter;
```



Created by HideMaru
from Noun Project

React Server Components

Back on page.jsx:

- Remove `import React from "react";` since it's not used here anymore.
- Import `ClickCounter` from its own file by adding this to the top of page.jsx:

```
import ClickCounter from "./ClickCounter";
```

Icon by HideMaru for Noun Project

File Edit Selection View Go Run ···

14 React Fundamentals

EXPLORER

- 14 REACT FUNDAMENTALS
  - react-01
  - react-02
  - react-03
  - react-04
  - react-05
  - react-06
  - react-07
  - react-08
  - react-09
    - .next
    - app
      - ClickCounter.jsx
      - page.jsx
    - node_modules
    - package-lock.json
    - package.json
  - react-custom
  - index.html

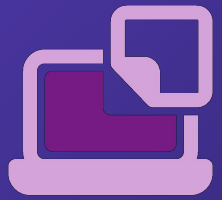
OPEN EDITORS

- page.jsx react-09\...

page.jsx | ClickCounter.jsx | package.json

react-09 > app > page.jsx > List

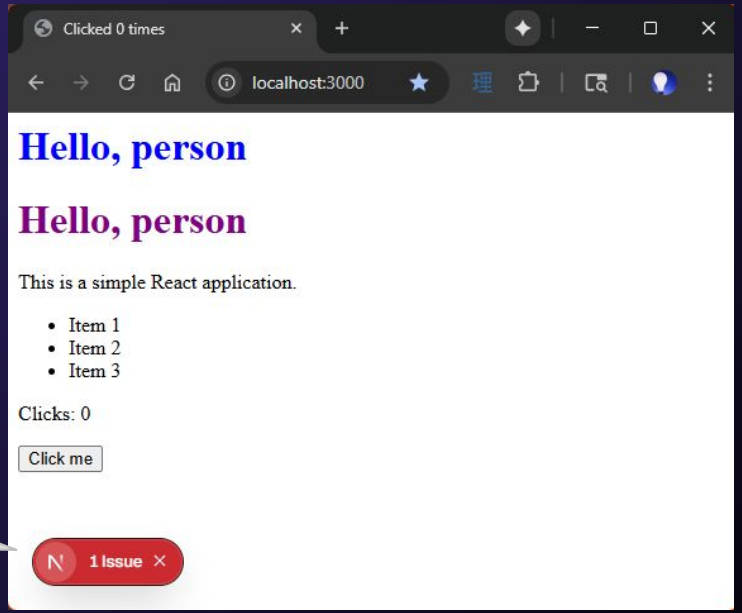
```jsx
import ClickCounter from "./ClickCounter";

const Title = ({ textColor = "purple" }) => {
    const name = "person";
    return <h1 style={{ color: textColor }}>Hello, {name}</h1>;
};
const Body = () => {
    return <p>This is a simple React application.</p>;
};
const List = () => {
    const items = ["Item 1", "Item 2", "Item 3"];
    return (
        <ul>
            {items.map((item, index) => (
                <li key={index}>{item}</li>
            ))}
        </ul>
    );
};
const HomePage = () => {
    return (
        <>
            <Title textColor="blue" />
            <Title />
            <Body />
            <List />
            <ClickCounter />
        </>
    );
};

export default HomePage;
```



Created by HideMaru
from Noun Project

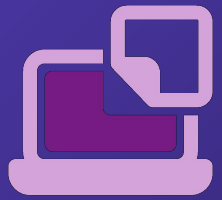
Finally running

Your application should be running again in localhost:3000.



There is “1 issue”. Click the bubble to get the error message.

Icon by HideMaru for Noun Project

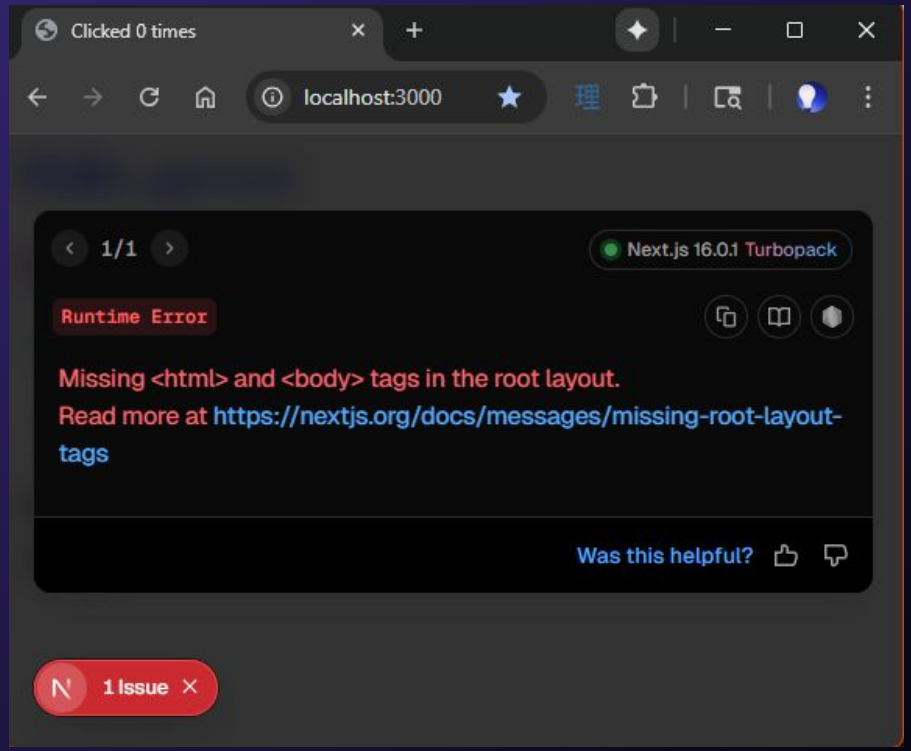


Created by HideMaru
from Noun Project

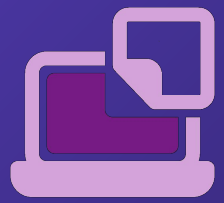
Layouts

This tells us the React app is rendered, but there is no <html> or <body> (remember we deleted them?)

Next has a special file for the HTML surrounding our app. Let's create it.



Icon by HideMaru for Noun Project



Created by HideMaru
from Noun Project

Layouts

- If it was not created automatically, make a new file within `/app` called `layout.tsx`
- Add the following to it:

```
export default function Layout({ children }) {
    return (
        <html>
            <body>{children}</body>
        </html>
    );
};
```

Here, children represents the app.
You can use this file to customize the static HTML surrounding the dynamic app in every page.

Clicked 0 times

localhost:3000

Hello, person

Hello, person

This is a simple React application.

- Item 1
- Item 2
- Item 3

Clicks: 0

Click me

Debugging

Now that we are using a proper React installation, to aid with debugging, we will install the React Developer Tools by following these instructions.
This will give you three more tabs in the Developer Tools:

- “Components” gives you a component tree. Now you can explore your code in the same way you explore the DOM tree.
- “Profiler” allows you to record the application for a limited time. You can then review how it behaves. This is useful to diagnose some problems and improve performance.
- “Suspense” helps you debug what components are blocking the page from displaying. It’s useful if your app is slow.

Clicked 0 times

localhost:3000

Hello, person

Hello, person

This is a simple React application.

- Item 1
- Item 2
- Item 3

Clicks: 0

Click me

Elements Console Recorder Components

Search (text or /regex/)

```
HeadManagerContext.Provider
  Root
    ServerRoot
      Context.Provider
        AppRouter
          RootErrorBoundary
            ErrorBoundary
              ErrorBoundaryHandler
                Router
                  HistoryUpdater
                  RuntimeStyles
                  NavigationPromisesContext.Provider
                    PathParamsContext.Provider
```

Initial Paint

suspended by

script

script

script

Clicked 0 times

localhost:3000

Hello, person

Hello, person

This is a simple React application.

- Item 1
- Item 2
- Item 3

Clicks: 0

Click me

html (in Layout) | 469px x 305px

Elements | Console | Recorder | Components

Search (text or /regex/)

Select an element in the page to inspect it

Use this tool to select components from the page

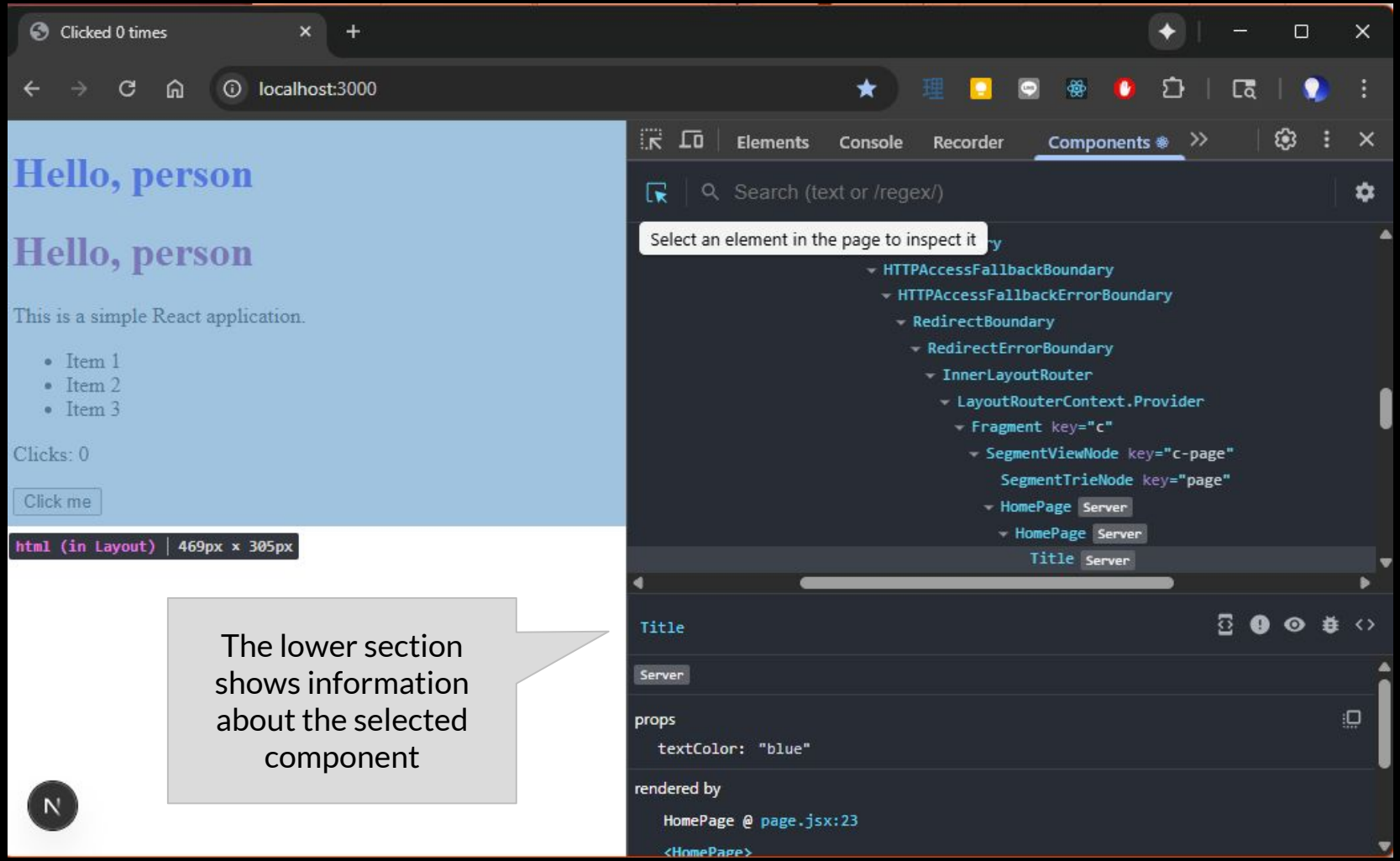
HTTPAccessFallbackBoundary
HTTPAccessFallbackErrorBoundary
ry
rBoundary
tRouter
terContext.Provider
t key="c"
tViewNode key="c-page"
entTrieNode key="page"
Page Server
HomePage Server
Title Server

Title

Server

props
textColor: "blue"

rendered by
HomePage @ page.jsx:23
<HomePage>


Clicked 0 times
localhost:3000
Hello, person
Hello, person
This is a simple React application.
Item 1
Item 2
Item 3
Clicks: 0
Click me
html (in Layout) | 469px x 305px
Elements
Console
Recorder
Components
Search (text or /regex/)
Select an element in the page to inspect it
HTTPAccessFallbackBoundary
HTTPAccessFallbackErrorBoundary
RedirectBoundary
RedirectErrorBoundary
InnerLayoutRouter
LayoutRouterContext.Provider
Fragment key="c"
SegmentViewNode key="c-page"
SegmentTrieNode key="page"
HomePage Server
HomePage Server
Title Server
Title
Server
props
textColor: "blue"
rendered by
HomePage @ page.jsx:23
The lower section shows information about the selected component

Using external packages

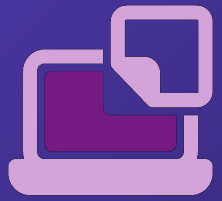
Before we continue, we want to explore how to extend React by using packages available in the npm *registry*.

Let’s look for a particular package that you may want to use later.

Using external packages

Before we continue, we want to explore how to extend React by using packages available in the npm *registry*.

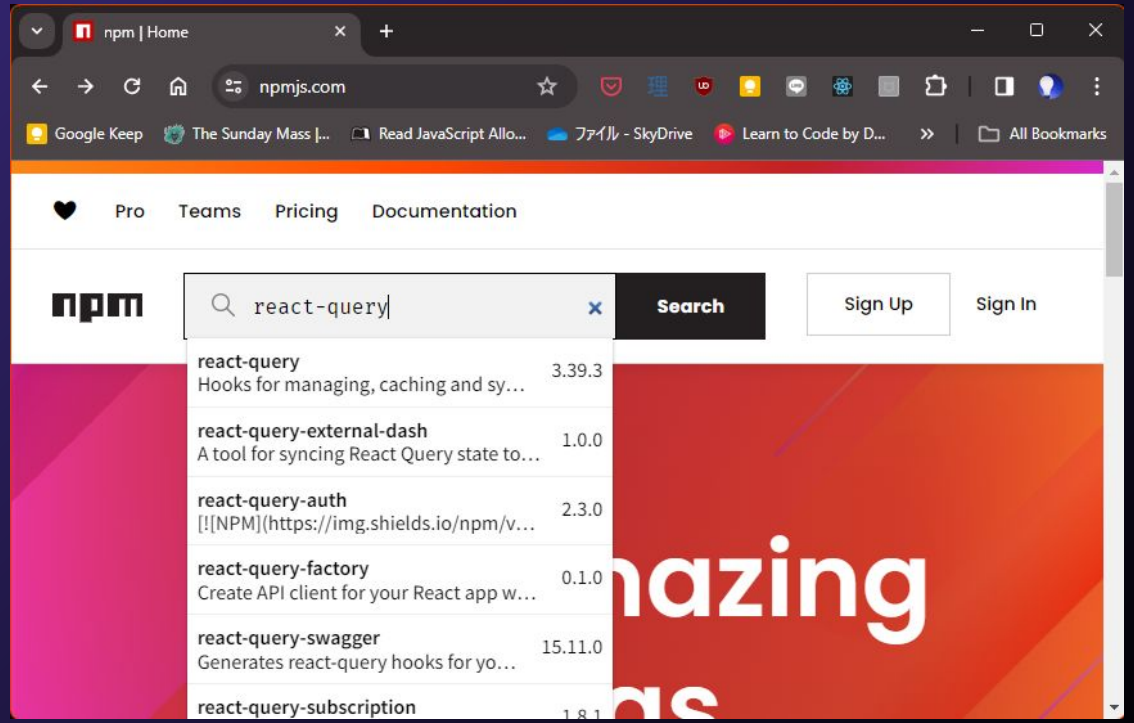
Let's look for react-query, a package that you may want to use to make it easier to get data from servers using React.



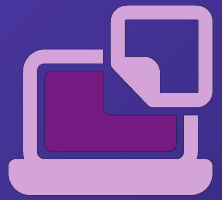
Created by HideMaru
from Noun Project

Using external packages

- Go to npmjs.com. In the search bar, look for “react-query”.



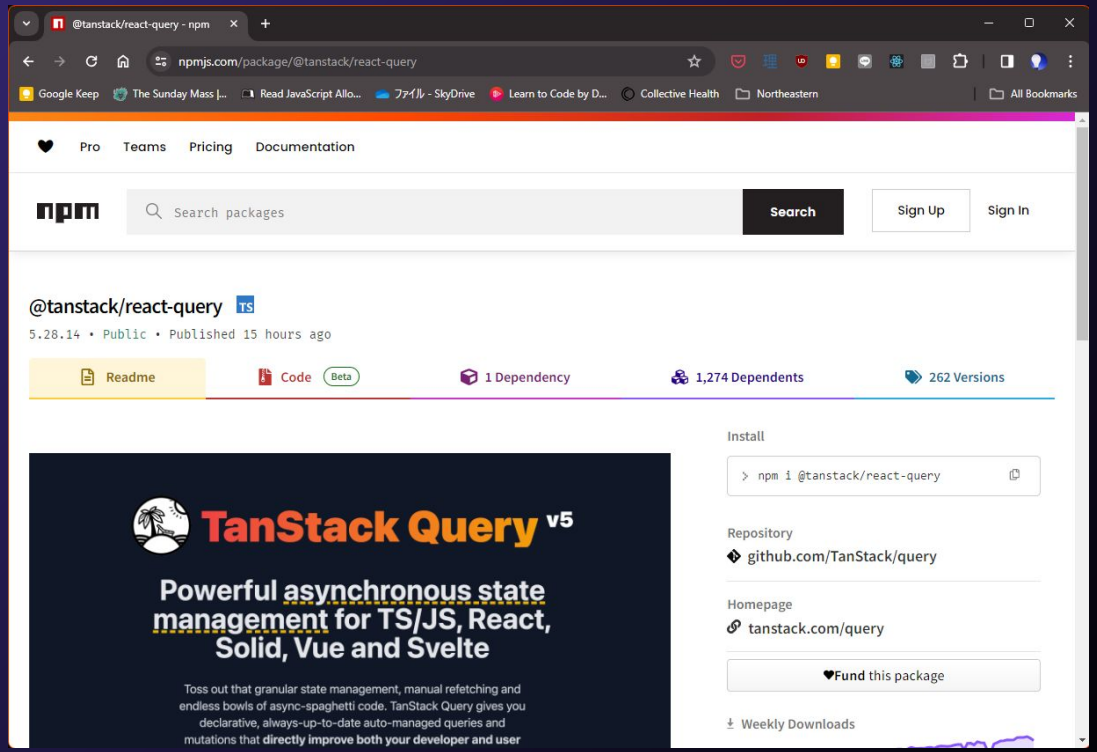
Icon by HideMaru for Noun Project



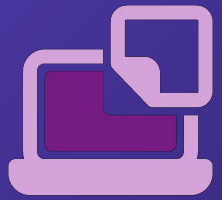
Created by HideMaru
from Noun Project

Using external packages

- Open the page for it. Note: you want to open: https://www.npmjs.com/package/@tanstack/react-query
- Read the page. Go to the project homepage to see if this is trustful and this is what you want.



Icon by HideMaru for Noun Project

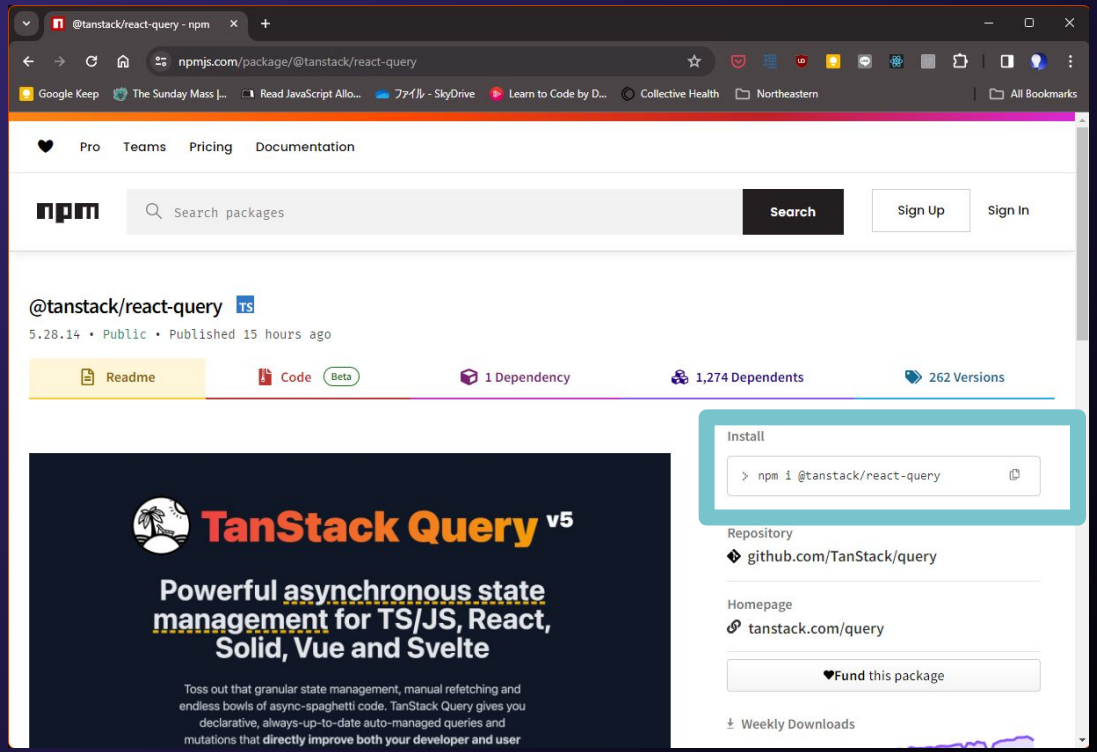


Created by HideMaru
from Noun Project

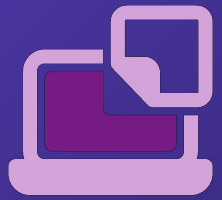
Using external packages

- Open the page for it. Note: you want to open: https://www.npmjs.com/package/@tanstack/react-query
- Search for the “Install” textbox and copy what’s in there:

```
npm i @tanstack/react-query
```



Icon by HideMaru for Noun Project



Created by HideMaru
from Noun Project

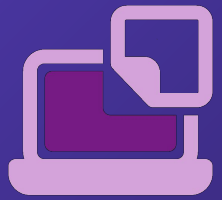
Using external packages

- Go now to your terminal and open the directory with your project .
  - If the web server is running, close it (press Ctrl+C in whatever window it's running).
- Make sure to create a new branch in your git project.
  - This way, if the package is not what you wanted, you can easily revert.
- Now, install the package using the instruction you got: `npm i @tanstack/react-query`

Windows PowerShell

```
PS C:\Users\fa_lo\projects\sample-react> git branch
* master
PS C:\Users\fa_lo\projects\sample-react> git branch install-react-query
PS C:\Users\fa_lo\projects\sample-react> git checkout install-react-query
Switched to branch 'install-react-query'
M       src/App.js
PS C:\Users\fa_lo\projects\sample-react> git branch
* install-react-query
  master
PS C:\Users\fa_lo\projects\sample-react> npm i @tanstack/react-query

added 2 packages, and audited 1563 packages in 4s

264 packages are looking for funding
  run `npm fund` for details

8 vulnerabilities (2 moderate, 6 high)

To address all issues (including breaking changes), run:
  npm audit fix --force

Run `npm audit` for details.
PS C:\Users\fa_lo\projects\sample-react>
```

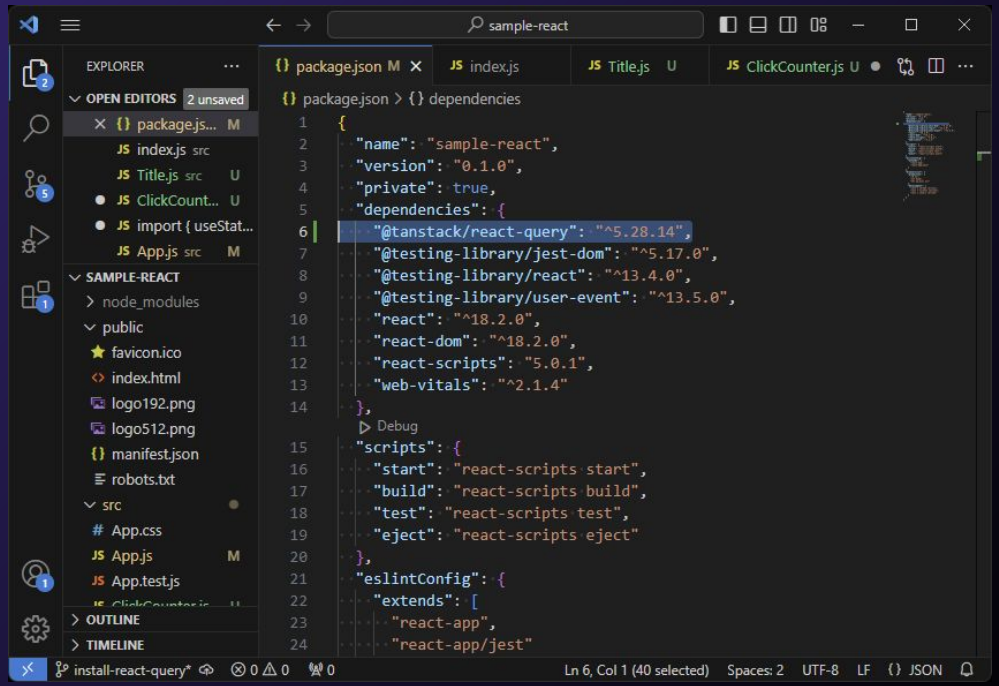
Icon by HideMaru for Noun Project



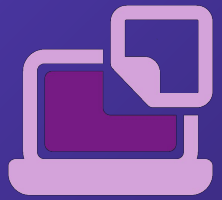
Created by HideMaru
from Noun Project

Using external packages

- What happened?
  - package.json got updated with the name and version of the installed package.
    - You will find it inside the “dependencies” section.
    - Whenever your app installs, npm will look for this version or newer (the “^” at the beginning of the number means “this version or newer”).



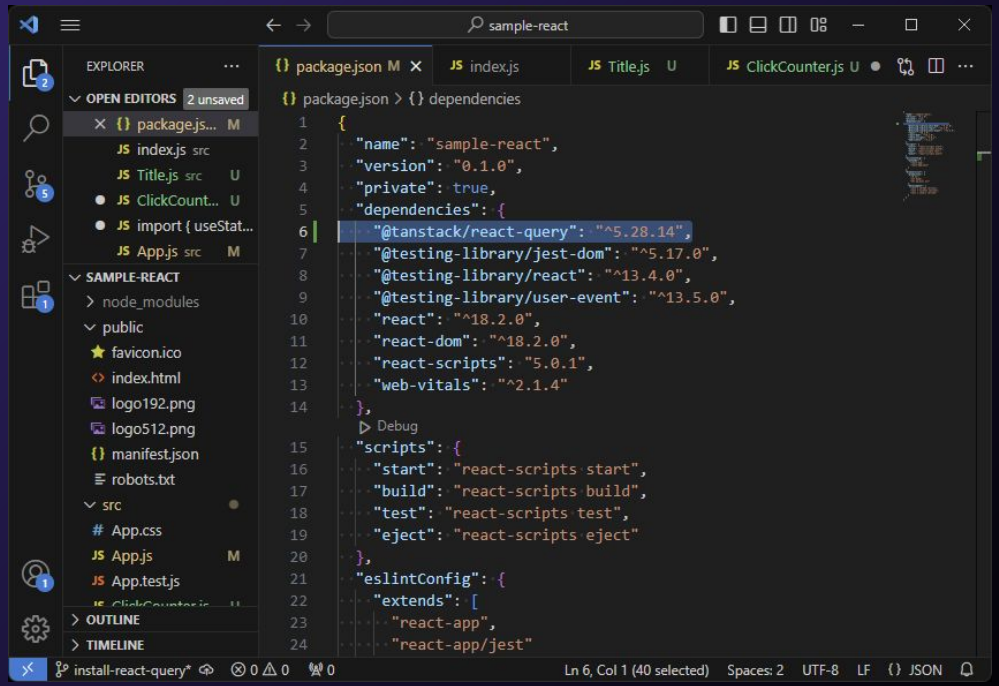
Icon by HideMaru for Noun Project



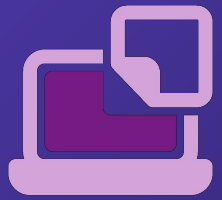
Created by HideMaru
from Noun Project

Using external packages

- What happened?
  - package.json got updated with the name and version of the installed package.
    - Some packages are installed with go into the “devDependencies” section instead. These are packages that are not included in the final version of your program; they’re only needed to build it (example: lint tools, etc.)



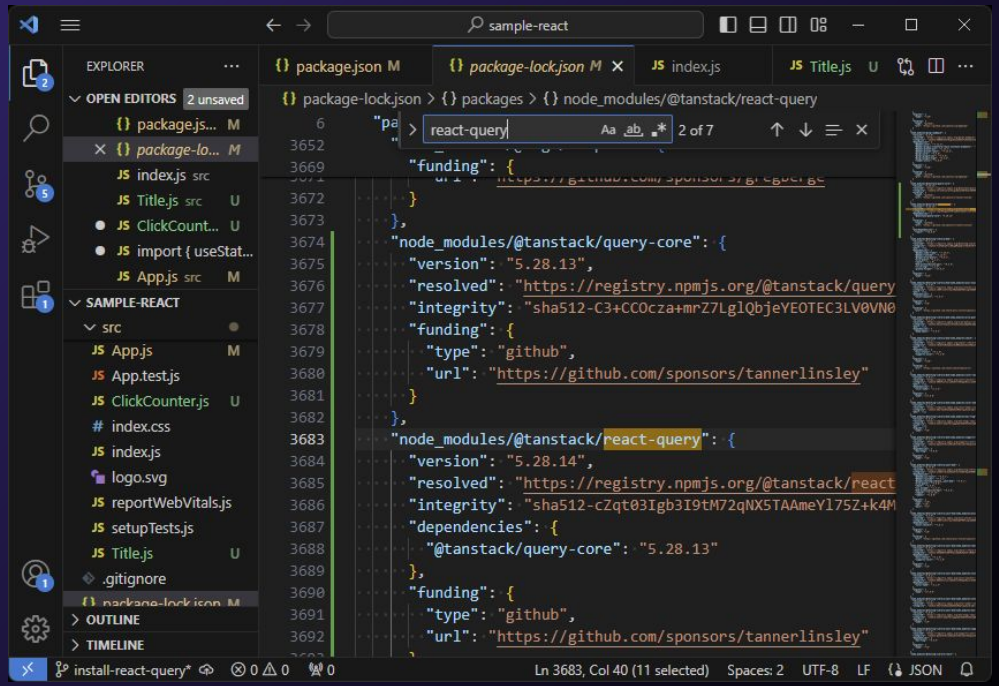
Icon by HideMaru for Noun Project



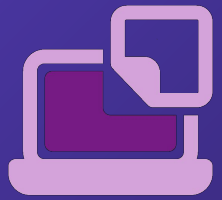
Created by HideMaru
from Noun Project

Using external packages

- What happened?
  - package-lock.json also got updated.
    - This file contains references to ALL of the packages that your packages need.
    - This is a big file, and it's complex!
  - Remember: DO NOT EDIT package.json OR package-lock.json BY HAND!
  - Note: if you're using a npm alternative (such as yarn) your files might have different names.



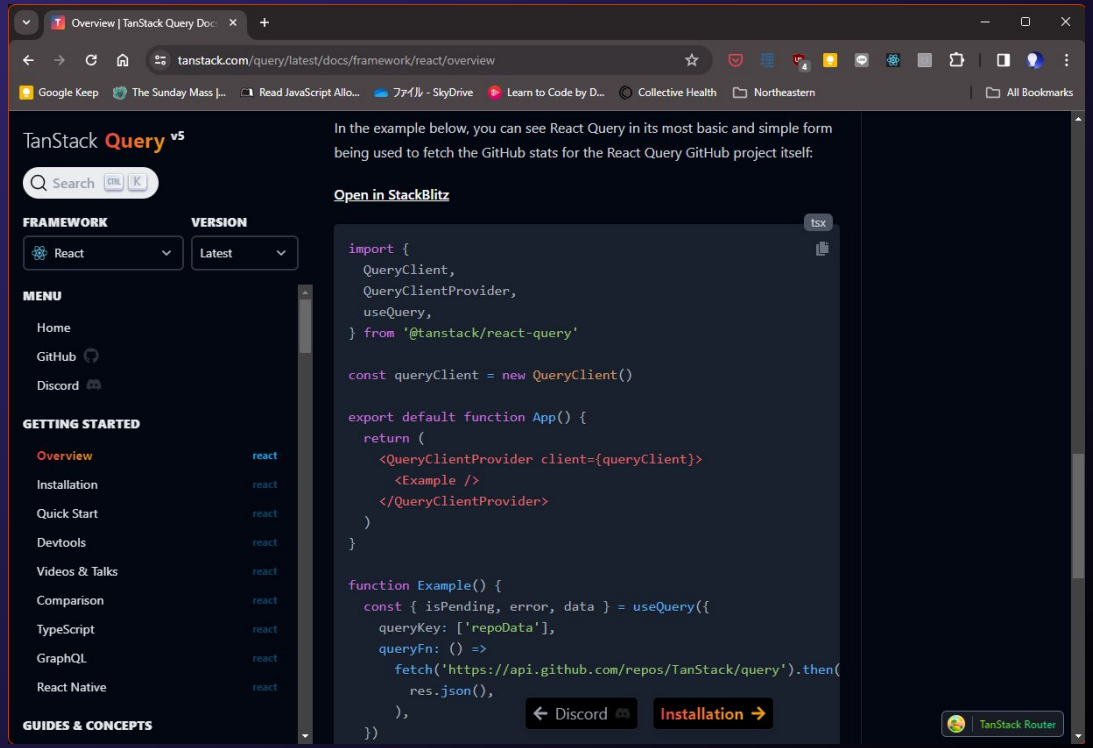
Icon by HideMaru for Noun Project



Created by HideMaru
from Noun Project

Using external packages

- How to use your new package?
  - Follow the instructions!
  - You will usually need to `import` something.
- react-query in particular gives you many new hooks. The simplest one is called `useQuery`.
- For a simple, real-world example of react-query with an actual API, see this: https://dev.to/otamnitram/react-query-a-practical-example-167j



Icon by HideMaru for Noun Project

Some alternatives for using API calls in React

...are in this tutorial, where you have some choices:

- Use fetch() within useEffect() (the simplest option),
- Install and use SWR, a package much simpler than react-query but also fully featured,
- Learn to use the very popular Axios package; this feels like fetch() but also works on Node (so, a good option to learn if you want to do full-stack using Node as your backend),
- Use a custom hook called react-fetch-hook, which also looks very simple,
- and there is also another example for react-query (it's popular!).

Considerations when creating an app that connects to a server

- Call the server as little as possible. Calls are expensive!
  - Even if the service is free, every call means the user has to wait. It may also cost them to download data, eg., if using the app on a mobile network
- Properly manage your call state
  - This means: displaying output only when it's ready; displaying "loading…" when the call is loading; displaying and handling errors if the call ends with an error.
  - There are many techniques to properly handle errors. Most packages already give you good default configuration (for example: retrying up to three times). Start with the defaults, then research to improve UX.
- Cache results whenever possible
  - To call the server less, and to make apps work faster.
  - However, make sure the data can be safely cached; in MOST cases you NEED to always have the latest version of the data

Appendix: Creating a new Next.js project from scratch (the “real” way)

- The next time you want to create a new Next.js project, just run in your console:

```
npx create-next-app@latest
```

- npx is a version of npm that downloads and then executes code.
- The `create-next-app` script is the recommended way for creating a new Next project. More information here.
  - create-next-app is a scaffolder.

Creating a project.

Windows PowerShell

```
PS C:\Users\fa_lo\Projects> npx create-next-app@latest
√ What is your project named? ... sample-react-next
√ Would you like to use TypeScript? ... No / Yes
√ Which linter would you like to use? » ESLint
√ Would you like to use Tailwind CSS? ... No / Yes
√ Would you like your code inside a `src/` directory? ... No / Yes
√ Would you like to use App Router? (recommended) ... No / Yes
√ Would you like to use Turbopack? (recommended) ... No / Yes
√ Would you like to customize the import alias (`@/*` by default)? ... No / Yes
Creating a new Next.js app in C:\Users\fa_lo\Projects\sample-react-next.

Using npm.

Initializing project with template: app-tw


Installing dependencies:
- react
- react-dom
- next

Installing devDependencies:
- @tailwindcss/postcss
- tailwindcss
- eslint
- eslint-config-next
- @eslint/eslintrc


added 329 packages, and audited 330 packages in 32s

136 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
Initialized a git repository.

Success! Created sample-react-next at C:\Users\fa_lo\Projects\sample-react-next

PS C:\Users\fa_lo\Projects>
```

The project options in detail

```
PS C:\Users\fa_lo\Projects> npx create-next-app@latest
√ What is your project named? ... sample-react-next
√ Would you like to use TypeScript? ... No / Yes
√ Which linter would you like to use? » ESLint
√ Would you like to use Tailwind CSS? ... No / Yes
√ Would you like your code inside a `src/` directory? ... No / Yes
√ Would you like to use App Router? (recommended) ... No / Yes
√ Would you like to use Turbopack? (recommended) ... No / Yes
√ Would you like to customize the import alias (`@/*` by default)? ... No / Yes
Creating a new Next.js app in C:\Users\fa_lo\Projects\sample-react-next.
```

Running the web server…

next-server (v15.5.2)

```
PS C:\Users\fa_lo\Projects\sample-react-next> npm run dev

> sample-react-next@0.1.0 dev
> next dev --turbopack

   ▲ Next.js 15.5.2 (Turbopack)
   - Local:        http://localhost:3000
   - Network:      http://10.5.0.2:3000

 ✓ Starting...
 ✓ Ready in 1109ms
```

Create Next App

localhost:3000

NEXT.JS

1. Get started by editing `src/app/page.js`.
2. Save and see your changes instantly.

Deploy now

Read our docs

Learn

Examples

Go to nextjs.org →

Appendix: More advanced hooks

In this appendix:

- The useContext hook
- Caching, the useMemo and the useCallback hooks

Reference code for this class:
- useContext: https://codesandbox.io/p/sandbox/dreamy-babbage-2yjnvy
- Caching, useMemo, useCallback: https://codesandbox.io/p/sandbox/td6zks

The context hook

Sometimes one, or many React components need to use state that lives in a parent that is very far away.

In this case, the natural thing to do is to “cascade” the state, sending it as parameters to the children, which in turns sends it to its children, and so on and so forth.

While necessary, this is ugly and adds too much unnecessary code.
- The children in the middle are stuck with parameters that they don’t use themselves; they’re just sending them to their children.

The context hook

Consider the following example. This has been greatly minimized; in real applications the depth of these calls can be very big:

ParentWithState.jsx

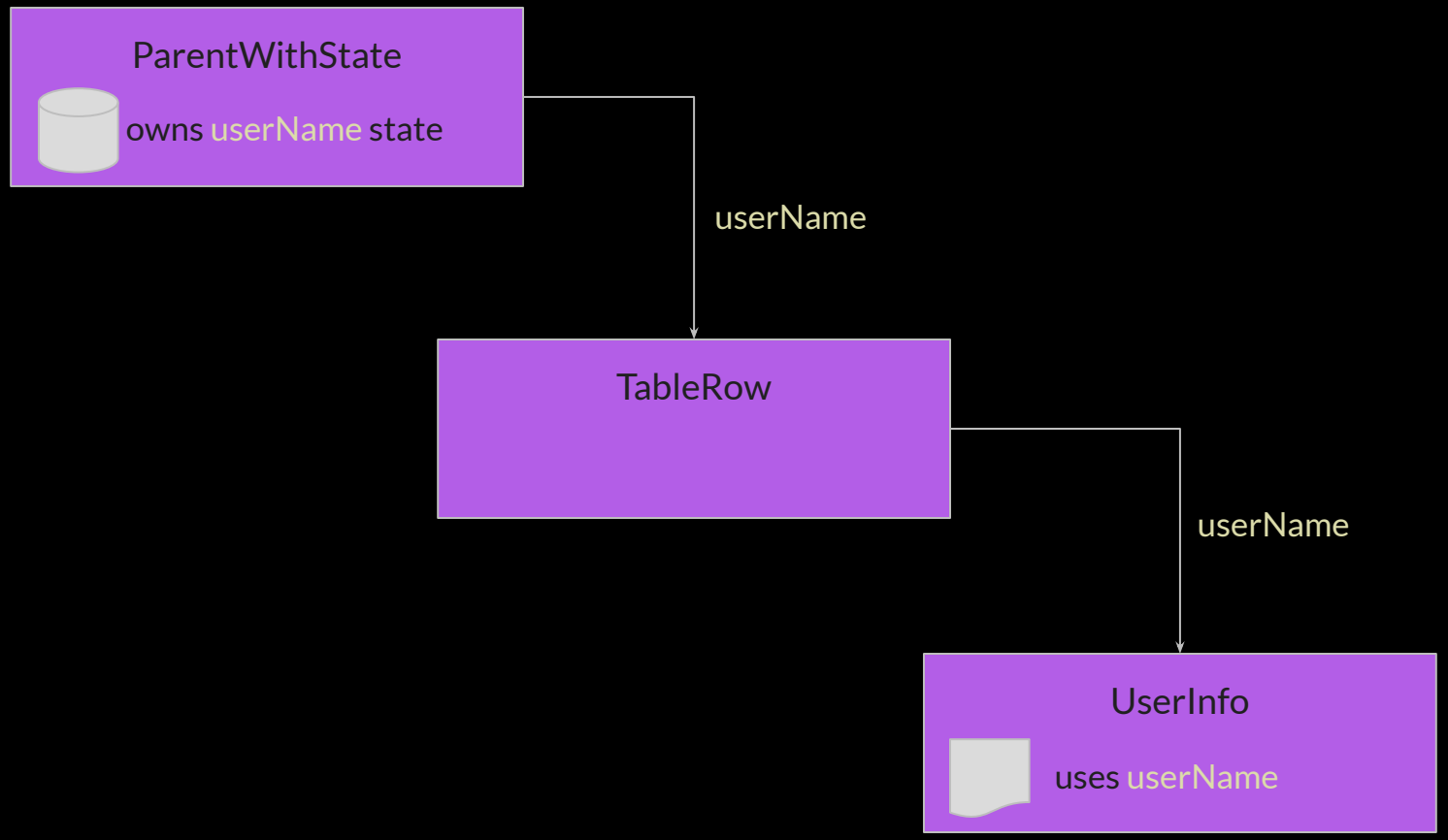
```
import React, { useState } from 'react';
const ParentWithState = () => {
    const [userName, setUserName] = useState('');

    /* ...other stuff happens... */

    return (
        <>
            <h1>Something</h1>
            <table>
                <TableRow userName={userName} />
            </table>
        </>
    );
}
export default ParentWithState;
```

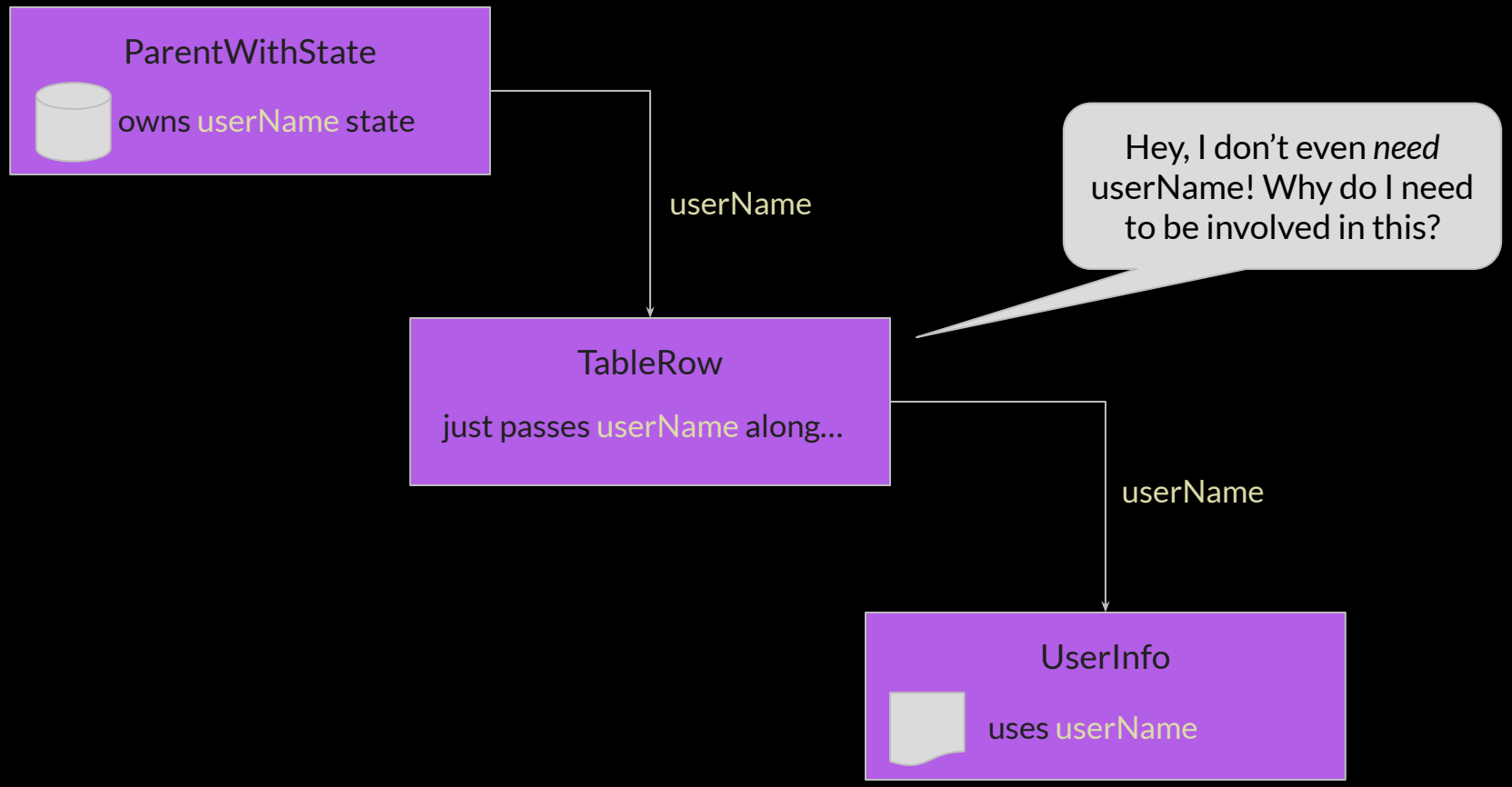
TableRow.jsx

```
import React from 'react';
export default const TableRow = ({ userName }) => {
    /* ...stuff... */
    return (
        <tr>
            <td>...</td>
            <td>
                <UserInfo userName={userName} />
            </td>
        </tr>
    );
}
export default TableRow;
```

UserInfo.jsx

```
export default const UserInfo = ({ userName }) => {
    /*...more stuff; probably search user info with this username*/
    return (
        <div className="someClass">
            <h2>{userName}</h2>
        </div>
    );
}; export default UserInfo;
```


ParentWithState
owns userName state
userName
TableRow
userName
UserInfo
uses userName

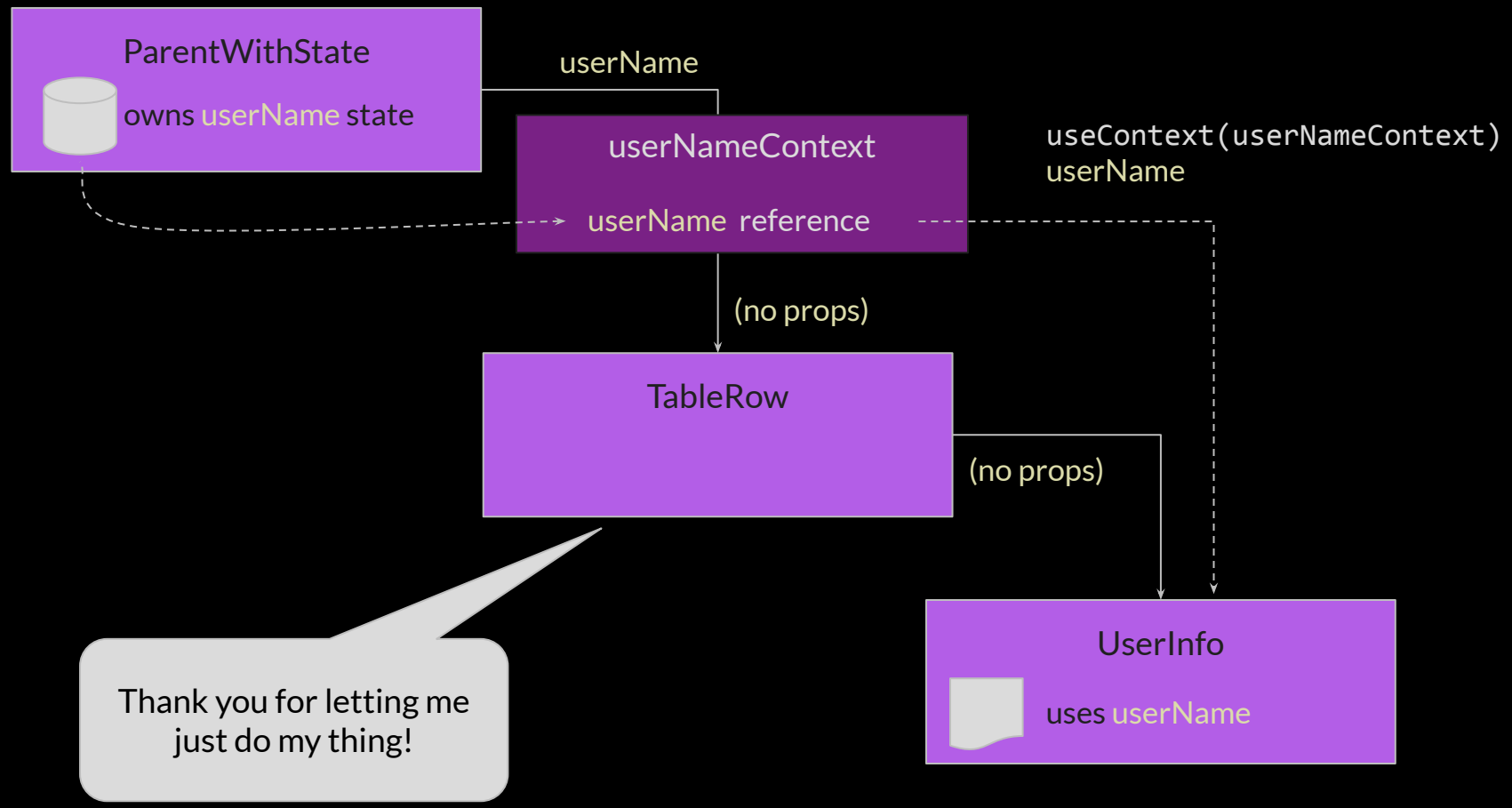

ParentWithState
owns userName state
userName
Hey, I don't even *need* userName! Why do I need to be involved in this?
TableRow
just passes userName along...
userName
UserInfo
uses userName

The useContext hook

To prevent the middle children to have to just pass parameters along, we can use the useContext hook.

useContext will create a “context” at the parent level. A context is a place where we can store references to state, so that any children can consume that state directly.

- This means: the last children can pull userName directly from the context; the intermediate children do not need to pass it down.


ParentWithState
owns userName state
userName
userNameContext
userName reference
useContext(userNameContext)
userName
(no props)
TableRow
(no props)
UserInfo
uses userName
Thank you for letting me just do my thing!

UserDataContext.jsx

```jsx
import { createContext } from 'react';

export const UserDataContext = createContext({
    userName: '',
});
```

TableRow.jsx

```jsx
export default const TableRow = () => {
    return (
        <tr>
            <td>...</td>
            <td>
                <UserInfo />
            </td>
        </tr>
    );
}
```

Note: `export default...` omitted for brevity

ParentWithState.jsx

```jsx
import React, { useState } from 'react';
const ParentWithState = () => {
    const [userName, setUserName] = useState('');
    return (
        <UserDataContext.Provider value={{ userName }}>
            <h1>Something</h1>
            <table>
                <TableRow />
            </table>
        </UserDataContext.Provider>
    );
}
```

UserInfo.jsx

```jsx
export default const UserInfo = () => {
    const { userName } = useContext(UserDataContext);
    return (
        <div className="someClass">
            <h2>{userName}</h2>
        </div>
    );
}
```

The useContext hook

What if we wanted to update the data in the context… from a children?

Yep, we can do that! We just send the update function in the context.

UserDataContext.jsx

```
import { createContext } from 'react';

export const UserDataContext = createContext({
    userName: '',
    setUserName: (_userName) => {},
});
```

TableRow.jsx

```
const TableRow = () => {
    return (
        <tr>
            <td>...</td>
            <td>
                <UserInfo />
            </td>
        </tr>
    );
}
```

ParentWithState.jsx

```
import React, { useState } from 'react';
const ParentWithState = () => {
    const [userName, setUserName] = useState('');
    return (
        <UserDataContext.Provider value={{ userName,
setUserName }}>
            <h1>Something</h1>
            <table>
                <TableRow />
            </table>
        </UserDataContext.Provider>
    );
}
```

UserInfo.jsx

```
const UserInfo = () => {
    const { userName, setUserName } = useContext(UserDataContext);
    const onChange = () => setUserName('admin');
    return (
        <div className="someClass">
            <h2>{userName}</h2>
            <button onClick={onChange} value="login as Admin"/>
        </div>
    );
}
```

UserDataContext.jsx

```
import { createContext } from 'react';

export const UserDataContext = createContext({
    userName: '',
    setUserName: (_userName) => {},
});
```

TableRow.jsx

```
const TableRow = () => {
    return (
        <tr>
            <td>...</td>
            <td>
                <UserInfo />
            </td>
        </tr>
    );
}
```

ParentWithState.jsx

```
import React, { useState } from 'react';
const ParentWithState = () => {
    const [userName, setUserName] = useState('');
    return (
        <UserDataContext.Provider value={{ userName,
setUserName }}>
            <h1>Something</h1>
            <table>
                <TableRow />
            </table>
        </UserDataContext.Provider>
    );
}
```

In reality this might come from a form with validation, etc.

UserInfo.jsx

```
const UserInfo = () => {
    const { userName, setUserName } = useCont(UserDataContext);
    const onChange = () => setUserName('admin');
    return (
        <div className="someClass">
            <h2>{userName}</h2>
            <button onClick={onChange} value="login as Admin"/>
        </div>
    );
}
```

The useContext hook

The useContext is ideal to store state that will be used by many children.

Prefer useContext over global state. Resist the temptation to just put a single context on the root and store everything there (unless it’s a very simple app).

Caching and the memo hook

Sometimes, we need to make operations on each render. These operations can be very expensive (this means: they take a lot of time). This can make our programs very slow, especially if we have to do them many times.

Consider a program that displays Fibonacci numbers. A Fibonacci number is defined by:

```
f(n) = f(n-1) + f(n-2)
```

where `f(0) = 0` and `f(1) = 1`.

For example:

- `f(2) = f(1) + f(0) = 0 + 1 = 1`
- `f(3) = f(2) + f(1) = f(1) + f(0) + f(1) = 0 + 1 + 1 = 2`

and so on.

Caching and the memo hook

Note that calculating any Fibonacci number necessarily needs the calculation of all the previous ones. Imagine, for example, calculating `f(100)`:

```
f(100) = f(99) + f(98)
       = f(98) + f(97) + f(98)
```

Caching and the memo hook

Note that calculating any Fibonacci number necessarily needs the calculation of all the previous ones. Imagine, for example, calculating `f(100)`:

```
f(100) = f(99) + f(98)
       = f(98) + f(97) + f(98)
       = f(97) + f(96) + f(97) + f(97) + f(96)
       = ...
```

You can probably already see how this operation is TOO EXPENSIVE. This is a lot of sums!

Caching and the memo hook

Note that calculating any Fibonacci number necessarily needs the calculation of all the previous ones. Imagine, for example, calculating `f(100)`:

```
f(100) = f(99) + f(98)
       = f(98) + f(97) + f(98)
       = f(97) + f(96) + f(97) + f(97) + f(96)
       = ...
```

You can probably already see how this operation is TOO EXPENSIVE. This is a lot of sums!

But also, we keep using the same numbers over and over.

Caching and the memo hook

We can probably make this easier if, instead of calculating the same f(n) over and over…

…once we know one, we write in a chart:

```
console.log(f(2))
```

```
f(2) = f(1) + f(0) = 1 + 0 = 1
```

n in f(n)	result
f(0)	0
f(1)	1
f(2)	1

Caching and the memo hook

And the next one we need it, instead of re-calculating it, we just check if the chart has it, and use it from there:

```
console.log(f(2)) -> 1
```

n in f(n)	result
f(0)	0
f(1)	1
f(2)	1

What is caching?

Caching means storing precalculated results.

This only works if the cached function is pure. In other words, the same parameters must always return the same output.

- The word “caching” in the context of a network call has the same meaning: when the browser requests a web resource, it saves it in memory; the next time the user requests it, it won’t pull it from the network again, but from the memory.

When caching is applied to functions it is also called memoizing: that is, to add memo(ry) to a function.

Caching and the memo hook

Back to React: The `useMemo` hook adds cache to a pure function.

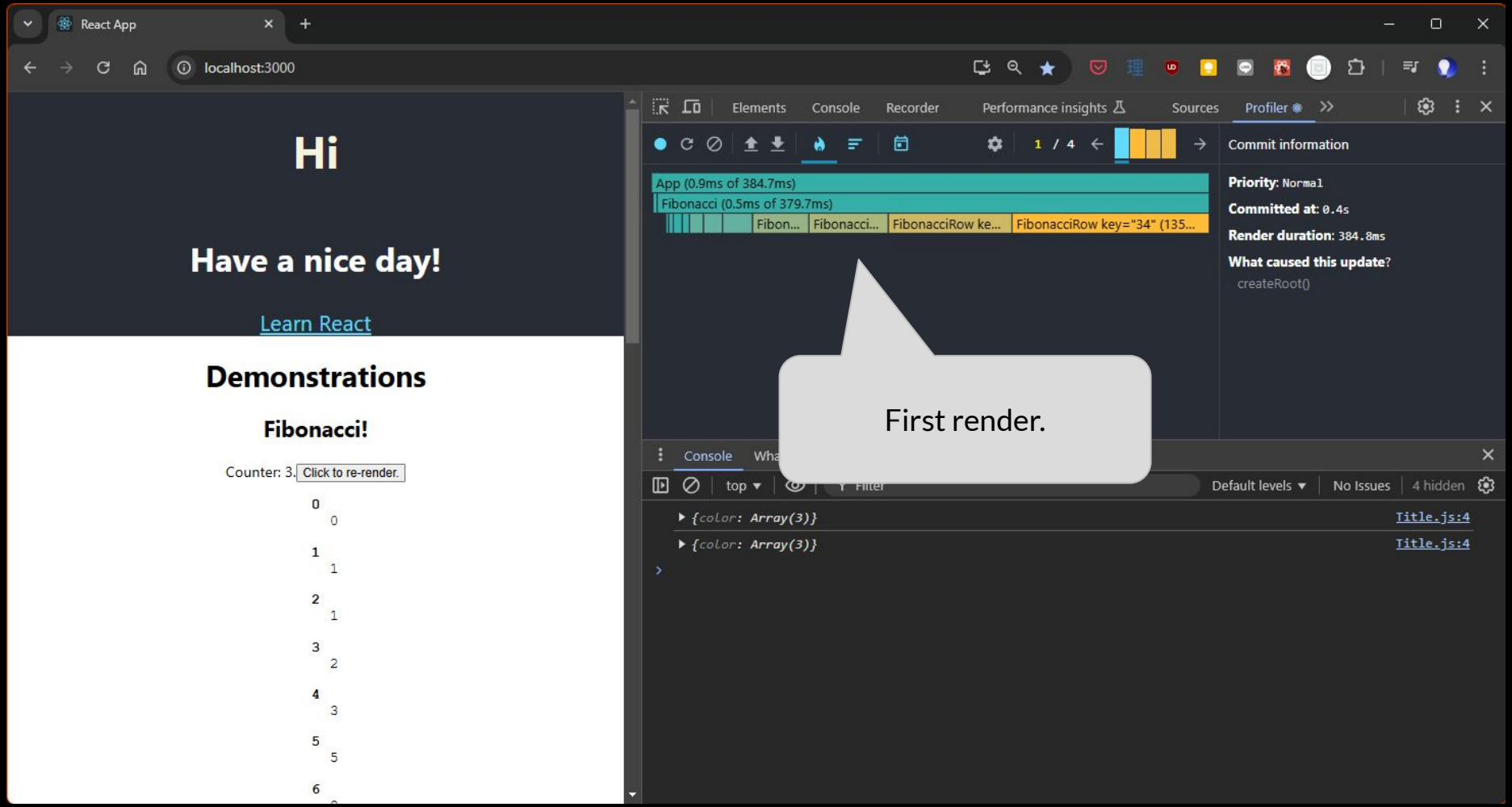
- It’s worth using it if the function is actually expensive: if it’s not, setting up useMemo can end up hurting performance. Do tests to figure out if useMemo is right for you.
- useMemo will preserve the value of the function across renders.

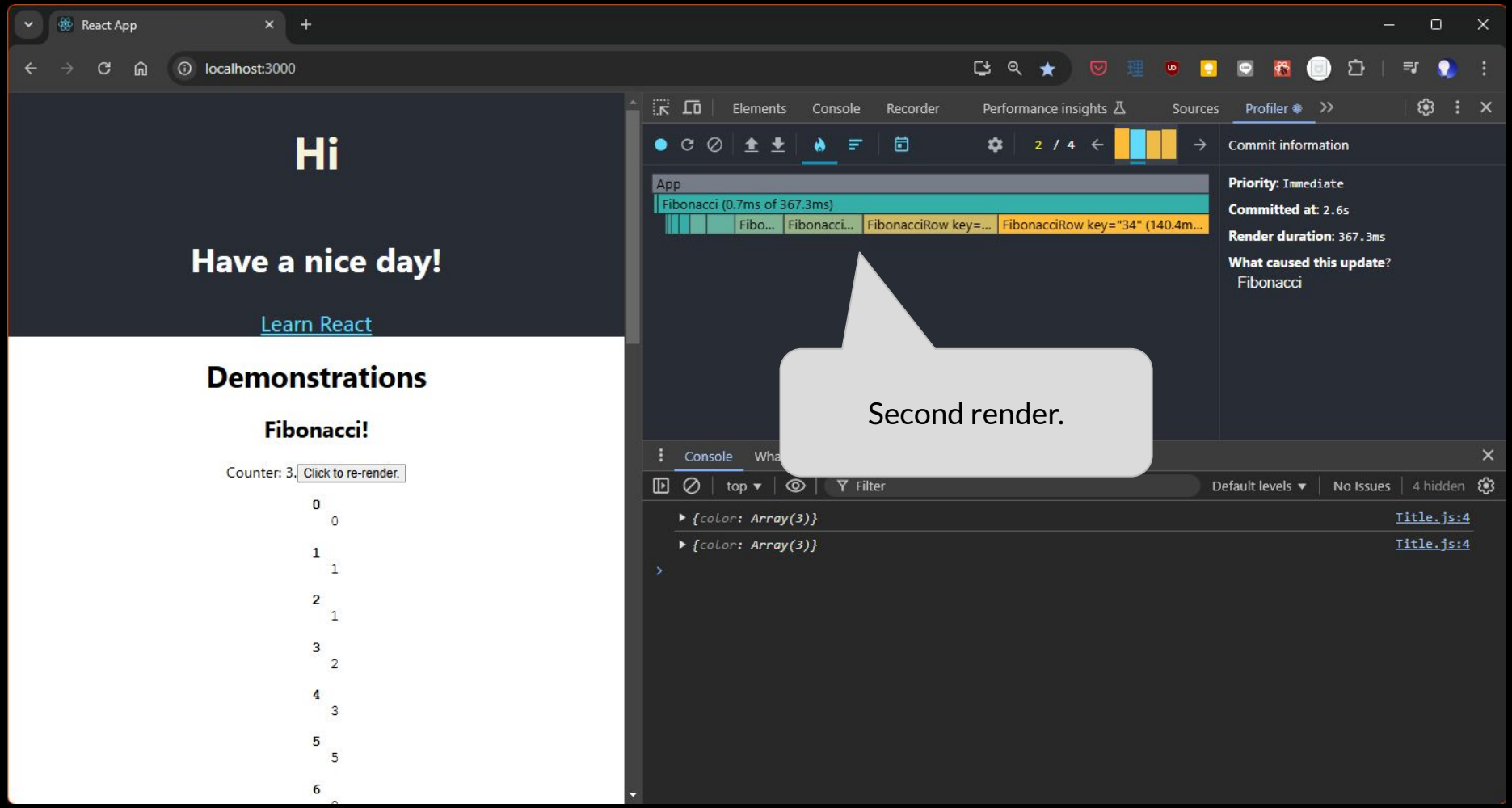
Caching and the memo hook

In the following example, I am forcing the app to calculate 35 Fibonacci numbers each time we render.

There is also a button that increases a counter (state). Increasing the counter changes state, so the component re-renders.

Because nothing is memoized, the 35 Fibonacci numbers are re-rendered each time. This is wasteful.


React App
localhost:3000
Hi
Have a nice day!
Learn React
Demonstrations
Fibonacci!
Counter: 3.
Click to re-render.
0
0
1
1
2
1
3
2
4
3
5
5
6
Elements
Console
Recorder
Performance insights
Sources
Profiler
1 / 4
Commit information
App (0.9ms of 384.7ms)
Fibonacci (0.5ms of 379.7ms)
Fibon...
Fibonacci...
FibonacciRow ke...
FibonacciRow key="34" (135...
Priority: Normal
Committed at: 0.4s
Render duration: 384.8ms
What caused this update?
createRoot()
First render.
Console
top
Default levels
No Issues
4 hidden
{color: Array(3)}
Title.js:4
{color: Array(3)}
Title.js:4


React App
localhost:3000
Hi
Have a nice day!
Learn React
Demonstrations
Fibonacci!
Counter: 3.
Click to re-render.
0
0
1
1
2
1
3
2
4
3
5
5
6
Elements
Console
Recorder
Performance insights
Sources
Profiler
2 / 4
Commit information
App
Fibonacci (0.7ms of 367.3ms)
Fibo...
Fibonacci...
FibonacciRow key=...
FibonacciRow key="34" (140.4m...
Priority: Immediate
Committed at: 2.6s
Render duration: 367.3ms
What caused this update?
Fibonacci
Second render.
Console
top
Filter
Default levels
No Issues
4 hidden
{color: Array(3)}
Title.js:4
{color: Array(3)}
Title.js:4

Fibonacci.jsx

```jsx
import React, { useState } from 'react';
import FibonacciRow from './FibonacciRow';

const Fibonacci = () => {
    const [counter, setCounter] = useState(0);

    // will recalculate with each render
    const items = new Array(35).fill(0).map((item, index) => (
        <FibonacciRow index={index} key={index}/>
    ));

    const handleClick = (() => {
        setCounter(counter => coun
    });

    return (
        <>
            <h2>Fibonacci!<
            <p>
                Counter: {counter}.
                <button onClick={handleClick}>Click to
re-render.</button>
            </p>
            <dl className='fibs'>
                {items}
            </dl>
        </>
    );
};
export default Fibonacci;
```

Creates an array, then fills it with items.

FibonacciRow.jsx

```jsx
import React from 'react';

// warning: VERY EXPENSIVE for large numbers
const calculateFibonacci = (n) => {
    if (n === 0) return 0;
    if (n === 1) return 1;
    return calculateFibonacci(n-1) +
calculateFibonacci(n-2);
}

const FibonacciRow = ({index}) => {
    const result = calculateFibonacci(index);
    return (
        <>
            <dt>{index}</dt>
            <dd>{result}</dd>
        </>
    )
};

export default FibonacciRow;
```

Fibonacci.jsx

```
import React, { useState } from 'react';
import FibonacciRow from './FibonacciRow';

const Fibonacci = () => {
    const [counter, setCounter] = useState(0);

    // will recalculate with each render
    const items = new Array(35).fill(0).map((item, index) => (
        <FibonacciRow index={index} key={index}/>
    ));

    const handleClick = (() => {
        setCounter(counter => counter + 1);
    });

    return (
        <>
            <h2>Fibonacci!</h2>
            <p>
                Counter: {counter}
                <button onClick={ha
re-render.</button>
            </p>
            <dl className='fibs'>
                {items}
            </dl>
        </>
    );
};
export default Fibonacci;
```

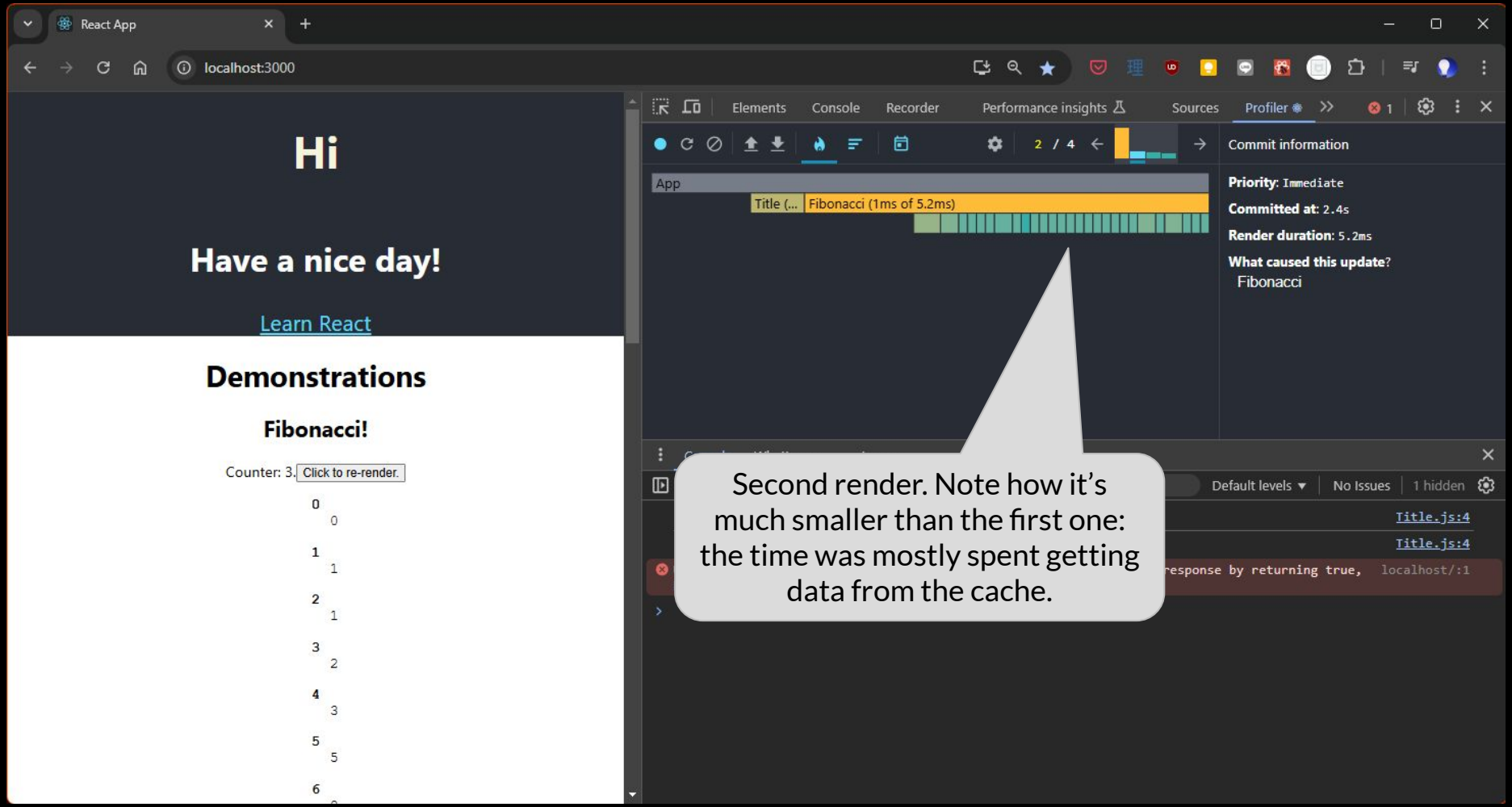
Functions that call themselves are called **recursive**.

FibonacciRow.jsx

```
import React from 'react';

// warning: VERY EXPENSIVE for large numbers
const calculateFibonacci = (n) => {
    if (n === 0) return 0;
    if (n === 1) return 1;
    return calculateFibonacci(n-1) +
calculateFibonacci(n-2);
}

const FibonacciRow = ({index}) => {
    const result = calculateFibonacci(index);
    return (
        <>
            <dt>{index}</dt>
            <dd>{result}</dd>
        </>
    )
};

export default FibonacciRow;
```

Caching and the memo hook

Now let’s apply useMemo to the calculating function and see what happens.


React App
localhost:3000
Hi
Have a nice day!
Learn React
Demonstrations
Fibonacci!
Counter: 3.
Click to re-render.
0
0
1
1
2
1
3
2
4
3
5
5
6
Elements
Console
Recorder
Performance insights
Sources
Profiler
2 / 4
Commit information
App
Title (...
Fibonacci (1ms of 5.2ms)
Priority: Immediate
Committed at: 2.4s
Render duration: 5.2ms
What caused this update?
Fibonacci
Default levels
No Issues
1 hidden
Title.js:4
Title.js:4
response by returning true,
localhost/:1
Second render. Note how it's much smaller than the first one: the time was mostly spent getting data from the cache.

Fibonacci.jsx

```
import React, { useState } from 'react';
import FibonacciRow from './FibonacciRow';

const Fibonacci = () => {
    const [counter, setCounter] = useState(0);

    // will recalculate with each render
    const items = new Array(35).fill(0).map((item, index) => (
        <Fibonacci Row index={index} key={index}/>
    ));

    const handleClick = (() => {
        setCounter(counter => counter + 1);
    });

    return (
        <>
            <h2>Fibonacci!</h2>
            <p>
                Counter: {counter}.
                <button onClick={handleClick}>Click to
re-render.</button>
            </p>
            <dl className='fibs'>
                {items}
            </dl>
        </>
    );
};
export default Fibonacci;
```

FibonacciRow.jsx

```
import React from 'react';

// warning: VERY EXPENSIVE for large numbers
const calculateFibonacci = (n) => {
    if (n === 0) return 0;
    if (n === 1) return 1;
    return calculateFibonacci(n-1) +
calculateFibonacci(n-2);
}

const FibonacciRow = ({index}) => {
    const result = useMemo(() =>
calculateFibonacci(index), [index]);

    return (
        <>
            <dt>{index}</dt>
            <dd>{result}</dd>
        </>
    )
};

export default FibonacciRow;
```

Caching and the memo hook

useMemo has two parameters:

- The first is a callback function to memoize. This cannot have parameters.
- The second is the list of parameters you used inside the callback function.

```
const result = useMemo(
    () => calculateFibonacci(index),
    [index]
);
```

The useCallback hook

Everything within a React component gets re-created when the component is re-rendered. This means, in this function, the handleClick function is constantly re-created:

```
const Fibonacci = () => {
    const [counter, setCounter] = useState(0);
    const items = new Array(35).fill(0).map((item, index) => (
        <FibonacciRow index={index} key={index}/>
    ));

    const handleClick = (() => {
        setCounter(counter => counter + 1);
    });
```

This is usually okay, but in some cases (eg. if `Fibonacci` itself was being memoized) we can make sure the function itself is preserved across renders.

The useCallback hook

Everything within a React component gets re-created when the component is re-rendered. This means, in this function, the handleClick function is constantly re-created:

```
const Fibonacci = () => {
    const [counter, setCounter] = useState(0);
    const items = new Array(35).fill(0).map((item, index) => (
        <FibonacciRow index={index} key={index}/>
    ));

    const handleClick = (() => {
        setCounter(counter => counter + 1);
    });
```

This is usually okay, but in some cases (eg. if `Fibonacci` itself was being memoized) we can make sure the function itself is preserved across renders.

The useCallback hook

To do so:

```
    const handleClick = useCallback(() => {
        setCounter(counter => counter + 1);
    }, [setCounter]);
```

This is similar to useMemo, except that it memoizes *the function itself*, not its return value. See [the React docs]() for more comprehensive examples.

Memoizing the helper function

What if we could actually add cache to the `calculateFibonacci` function?

We can manually memoize the function to use its own internal cache.

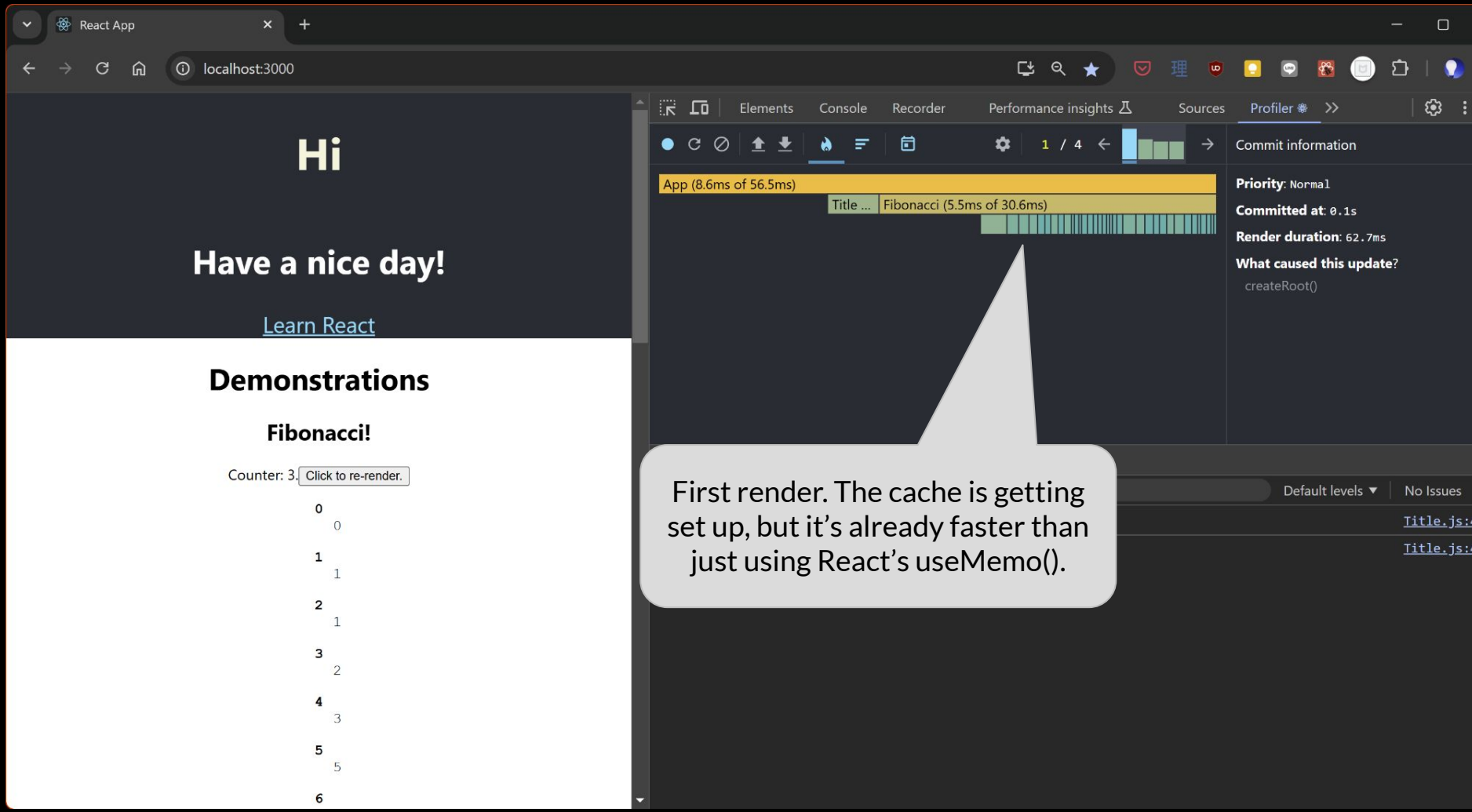
```
f(3) -> f(2) + f(1) = 1 + 1 = 2
```

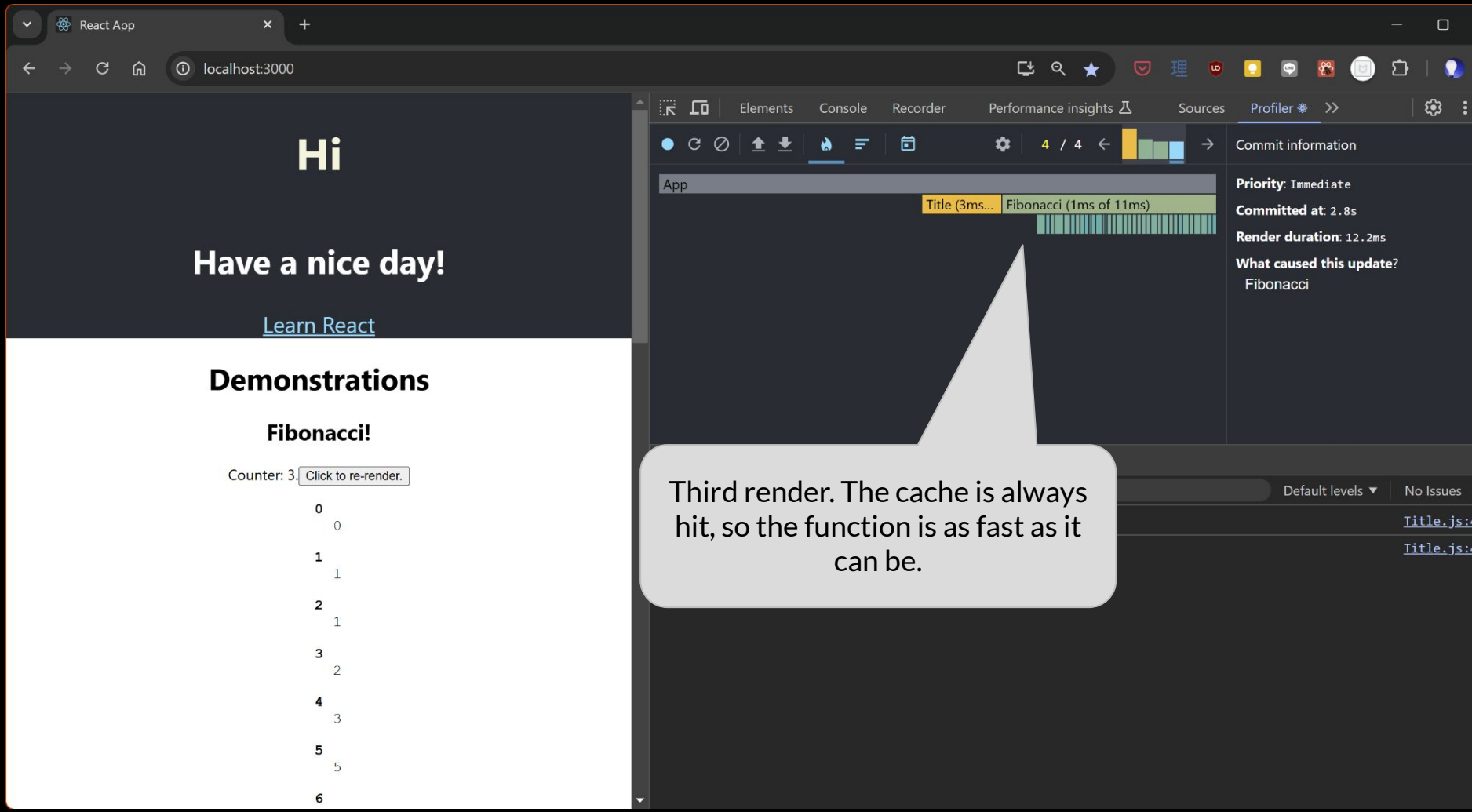
n in f(n)	result
f(0)	0
f(1)	1
f(2)	1
f(3)	2

We can do this manually, inside of the function:

FibonacciRow.jsx

```
import React, { useMemo } from 'react';

const calculateFibonacci = (n, cache) => {
    if (n === 0) return 0;
    if (n === 1) return 1;
    if (cache[n]) {
        return cache[n];
    }
    cache[n] = calculateFibonacci(n-1, cache) + calculateFibonacci(n-2, cache);
    return cache[n];
}

const FibonacciRow = ({index}) => {
    const result = useMemo(
        () => {
            const cache = [];
            return calculateFibonacci(index, cache);
        },
        [index]
    );
    return (
    ...
```


React App
localhost:3000
Hi
Have a nice day!
Learn React
Demonstrations
Fibonacci!
Counter: 3.
Click to re-render.
0
0
1
1
2
1
3
2
4
3
5
5
6
Elements
Console
Recorder
Performance insights
Sources
Profiler
1 / 4
Commit information
App (8.6ms of 56.5ms)
Title ...
Fibonacci (5.5ms of 30.6ms)
Priority: Normal
Committed at: 0.1s
Render duration: 62.7ms
What caused this update?
createRoot()
Default levels
No Issues
Title.js:
Title.js:
First render. The cache is getting set up, but it's already faster than just using React's useMemo().


React App
localhost:3000
Hi
Have a nice day!
Learn React
Demonstrations
Fibonacci!
Counter: 3.
Click to re-render.
0
0
1
1
2
1
3
2
4
3
5
5
6
Elements
Console
Recorder
Performance insights
Sources
Profiler
4 / 4
Commit information
App
Title (3ms...
Fibonacci (1ms of 11ms)
Priority: Immediate
Committed at: 2.8s
Render duration: 12.2ms
What caused this update?
Fibonacci
Default levels
No Issues
Title.js:
Title.js:
Third render. The cache is always hit, so the function is as fast as it can be.

Memoizing the helper function

Usually you won't have to write memoizers manually. You can use pre-made helper functions, such as memoize, which is part of the popular Lodash package.

Lodash is a utility library that contains many other helper functions for common problems; check it out!